OWNER-AUTHORIZED / PRE-RELEASE / FROZEN SOURCE

Groupmine 2.2.0

Independent Internal Security Review

Complete English report

RELEASE RECOMMENDATION

Hold production custody

Resolve deployment-identity gaps, known compiler exposure and the remaining candidate-specific release gates.

FROZEN COMMIT

`6345de8c488f3a7f2193a0fc8c067067334278ef`

REVIEWER

ChatGPT, GPT-6 Astra Pro

REVIEW DATE

28 October 2026 UTC / 29 October 2026 UTC+08

PDF EDITION

29 October 2026 / Asia-Shanghai

SCOPE

Production, setup/tooling, canary, dependencies and policy

Frozen implementation unchanged. No production credentials accessed. No public-chain transaction signed or broadcast. This internal review is not SALUS approval.

Contents

Complete review, including evidence limits and release gates. Section entries and PDF bookmarks are clickable.

1. Executive assessment . . . 3

2. Scope, target identity and evidence conventions . . . 4

3. Independent accounting derivation . . . 7

4. Lifecycle, authorization and custody boundary . . . 11

5. Findings register . . . 15

6. Production-contract findings . . . 16

7. Setup, deployment and tooling findings . . . 21

8. SALUS #1-#8 reassessment . . . 24

9. Integration, generated artifacts and separate canary scope . . . 26

10. External dependencies and current read-only observations . . . 29

11. Intended policies and remaining operational risks . . . 33

12. Fresh execution ledger and evidence limits . . . 34

13. Minimal correction priorities and release gates . . . 39

Appendix A. Evidence bundle and reproduction map . . . 41

Edition note: This PDF preserves the complete report dated 28 October 2026 UTC (29 October in UTC+08). Layout, navigation and the lifecycle diagram have been rendered for reading; this conversion adds no audit execution or new findings.

Evidence companion: GROUPMINE_6345de8_REVIEW_EVIDENCE.zip. Detailed results, proof sources, failed-attempt records and the SHA-256 manifest remain in that separate archive.

Report identity and scope

Reviewer: ChatGPT, GPT-6 Astra Pro, with independently delegated source-review workstreams.

Report date: 28 October 2026 (UTC; 29 October in UTC+08).

Frozen target: `6345de8c488f3a7f2193a0fc8c067067334278ef`.

Application: `TapeoutMinerPool/2.2.0`; package `2.2.0-rc.1`.

Owner authorization: First-party, pre-release source review and isolated local tests, as supplied in the attached brief.

Implementation disposition: No proposed fix was applied to the frozen implementation.

1. Executive assessment

Release recommendation: hold the custody release until the deployment-identity gaps and known compiler exposure are resolved and the remaining deployment-specific gates in section 13 are satisfied. This is an evidence-bounded security review, not a certification of safety and not SALUS approval.

The central accounting remediation is substantially sound under its stated input and token assumptions. Each incoming miner's actual measured payment is added to a persistent category ledger using that miner's own burn. Old registered miners are settled before composition changes. For the **same realized per-miner** `(received, nandBurn, latchBurn)` **inputs**, carried allocations are invariant under miner permutation and transaction partition. The required 105-BEM example yields NAND 150 and LATCH 50; subsequent ordinary income uses aggregate 15:5. Cumulative fees include carried income once. I found no demonstrated critical/high production-contract theft, insolvency, double-recognition, decreasing-entitlement, or unauthorized receipt-income-claim defect in the examined paths. This negative result is bounded by the source, execution and dependency limits recorded below.

There are nevertheless material findings:

1. **Deployment verification can omit the actual bound distributor.** An addresses file without `distributor` passes the schema and can produce `MATCHES`; the checker does not even read `pool.distributor()` on that path. Admission planning does not require that manifest field. This leaves a hole in the operational safeguard used to justify trusting the Pool's view-based distributor binding. **Medium, tooling.**
2. **Runtime masking does not establish immutable-value consistency.** The verifier ignores every constructor-immutable occurrence independently, then checks public getters. The exporter discarded the grouping that identifies repeated occurrences of the same immutable. Arbitrary differences inside those slots pass `compareRuntime`; a getter alone does not establish that operational call sites contain the same address. **Medium, deployment verification; the exact execution boundary of the defensive probe is stated in TL-02.**
3. **The high-precision ledger does not eliminate all same-ratio checkpoint effects.** Recognizing 63 raw units at burn 1:2 in one ordinary period gives NAND 20/LATCH 40. Recognizing 1, admitting a zero-carry miner with the same ratio, then recognizing 62 gives 19/40. A second counterexample refutes the documentation's literal universal “within one raw unit” rational-fairness claim. Conservation still holds. **Low, precision-policy/documentation qualification.**
4. **The remaining SALUS setup issues are still present.** Wallet revocation is unbounded; ownership rotation can separate retraction authority from the top-ID receipts; and a genuine outside receipt holder can delay allocation finalization by funding additional unpaid inventory.

The new activation gate propagates the last delay to custody opening. These are not closed merely because the remediation excludes them.

5. **Admission availability has a finite practical horizon.** Every new admission settles the complete old registry. One persistent non-REVOKED failure blocks new admissions, and growth eventually encounters gas limits. The author's real-protocol capacity evidence sampled 24 miners, not hundreds.
6. **The pinned compiler has an unresolved known-bug exposure.** The official SOL-2026-7 advisory includes Solidity 0.8.24 with this candidate's optimized via-IR settings. No candidate-specific miscompilation was demonstrated or excluded. The upstream rating is Low; for this unreleased permanent-custody application it is a separate release gate, not a newly proved production exploit. See EX-03.

Two further qualifications materially affect interpretation:

- The Pool's genuine receipt implementations are authenticated by exact runtime hash. Its distributor is authenticated only by reported views on-chain. A setup owner can bind a lying or upgradeable look-alike whose `owner()` is zero but which retains another privileged path. This is a setup-authority boundary, not an unprivileged takeover of an already genuine immutable distributor. The two tooling defects above matter because the documentation assigns identity enforcement to tooling.
- A fixed timestamp does not guarantee equal actual carried inputs. The retained Mining analysis describes a shared remaining-supply cap. Near that cap, claim order can change the realized per-miner payment vector. The Pool cannot make unequal inputs equal; its order-independence statement must explicitly preserve actual inputs and upstream state.

Fresh external observation that supersedes one historical assumption: at block 128,713,128 the NFT beacon reports the recorded nonzero owner, but at the newly sampled block 130,348,784 it reports `owner() == address(0)`. The beacon and all sampled dependency code hashes are unchanged. Therefore the retained assertion of a currently nonzero beacon owner must not be repeated as a present fact. Section 10 distinguishes historical authority, the new observation, and the scope of bytecode reasoning.

2. Scope, target identity and evidence conventions

2.1 Revisions and preservation

Table 1. Target revisions and provenance

Item	Revision / result
Repository	tapeminecoop/groupmine
Frozen primary target	`6345de8c488f3a7f2193a0fc8c067067334278ef`
Merge first parent / pre-remediation baseline	`c78a6d1234441137f8407fd30f904061ba792510`
SALUS's actual audited revision	`c3cb93375737de7ec63baa9f390edd0c15a5c7a4`

Item	Revision / result
Prior 2.1.0 candidate	`8cf31dc97e8f6c88bfdf9a983089bceda8baadcb`
`origin/main` at repository acquisition	Same as frozen target; no newer SHA was substituted
Author-run source fingerprint	`c464fe4bc89a9e1fd0de0943577458980b8231d2a33e6594b83373b442d87874`, independently reproduced across 163 files
SALUS PDF SHA-256	`2ecddb8f2b6ebdf3fda56dd6b91dd71e7e68155e389eef03fc3247794c9b8b7a`

Git identifies the primary target as a merge with the exact supplied first parent. The complete target was checked out detached and retained separately from an execution copy created by `git archive`. Audit additions, build outputs and logs were written outside the frozen checkout. Existing source, tests, lockfiles, historical reports and approval records were preserved. The final check recomputed all 667 tracked Git blobs and found no mismatch; Git status was clean. It also compared 122 original implementation, test, SDK/CLI/script and configuration inputs in the execution copy with the frozen checkout, finding no changed input. All 431 baseline files under the protected historical paths have unchanged Git blob IDs at the target. All six SALUS appendix plain file SHA-1 values match `c3cb933`; they are not Git blob IDs. Evidence: `provenance-hashes.json`, `target-identity.txt`, `baseline-diff-stat.txt`, and `source-preservation-final.json` in the evidence bundle.

The baseline diff changes 130 files, dominated by newly retained reports. The material production surface is `PoolMath.sol`, `TapeoutMinerPool.sol` and the new `IReceiptDistributor.sol`. Receipt and distributor source are unchanged. Pool imports change distributor compiler metadata even though its source logic is unchanged. SDK math/state readers, CLI finality/binding paths, generated artifacts, gas allowances, canary generation and adapted tests/reference models also changed. `package-lock.json` changes only the package version; dependency versions remain pinned.

2.2 Labels used throughout

Table 2. Evidence classifications

Label	Meaning
VERIFIED_BY_EXECUTION	An action actually run in this review session; each claim identifies whether it was Python arithmetic, an SDK fixture, local EVM execution, compilation, hashing, or public read-only RPC.
SOURCE_CONFIRMED	A conclusion established by reading the frozen source or a stated derivation from it, with assumptions made explicit. It does not imply an executed PoC.
AUTHOR_RECORDED	Retained developer/auditor reports, logs, assertions or observations not independently repeated here. A matching hash links them to the tree but does not convert them into this review's execution.
UNVERIFIED / NOT_EXECUTED	A check not completed, missing a necessary input, or outside the executed scope.
HYPOTHESIS	A candidate risk not established by source reasoning or execution; not counted as a demonstrated bug.

Severity reflects impact plus prerequisites. A malicious setup-owner scenario is not assigned the same threat model as an attack by an arbitrary runtime user. An external authority or an intentional irreversibility policy is identified separately from a defect in the reviewed production logic.

2.3 Files and methods

The review read all primary production Solidity: `src/TapeoutMinerPool.sol`, `src/PoolMath.sol`, `src/TapeoutEntitlementReceipt.sol`, `src/TapeoutReceiptDistributor.sol`, `src/interfaces/IMining.sol`, and `src/interfaces/IReceiptDistributor.sol`. It inspected the exact locked OpenZeppelin 5.4.0 ownership, ERC-1155 update/acceptance, reentrancy and SafeERC20 transfer implementations, including their relevant utility paths. Package-lock integrity was used for dependency installation; lifecycle scripts were disabled.

Context reviewed includes the original SALUS PDF; `docs/SALUS_AUDIT_REMEDIATION_REPORT.md`, `ACCOUNTING.md`, `ARCHITECTURE.md`, `OPERATIONS.md`, `MINING_LIFECYCLE.md`, relevant prior remediation/verification material; the retained SALUS run manifest, assertions, identities, fork/gas and canary observations; and the production/canary adaptation. Tooling review covered `sdk/math.ts`, `pool.ts`, `deployment.ts`, `config.ts`, `allocation.ts`, `verify.ts`, `tx.ts`, `gas.ts`, `constants.ts`, generated ABI/bytecode, `cli/pool.ts`, deployment tooling and relevant run scripts. Tests were inspected by purpose, including SALUS, finality invariants, FFI models, fork boundaries, unit/invariant, prior-audit and SDK regression suites. This is not a claim that every historical log was line-by-line re-audited or that the external Mining implementation received a complete formal proof.

Methods included baseline/source comparison, authority and call-boundary reasoning, an independent rational oracle separated from a candidate-precision oracle, deterministic small-state and randomized arithmetic checks, defensive SDK checker probes, fresh contract regressions, existing test reruns, source/build identity checks and read-only public dependency snapshots. The execution ledger supplies the final outcome of each attempted run.

2.4 Authorized boundaries observed

No production `.env`, private key, wallet session or user credential was read. No public-chain transaction was signed or broadcast. Public-chain operations were reads. Local tests used in-process EVM instances or verified loopback development nodes. FFI was restricted to inspected local Python arithmetic models. Live canary, preparation, approval and recovery entry points were not run. Optional deployment scripts containing broadcast hooks were read, not executed.

The application candidate is unreleased. Historical mainnet canary activity used the owner's controlled assets and an older economic model; this report neither denies that activity nor treats it as validation or authorization for this candidate.

3. Independent accounting derivation

3.1 Two oracles, two different questions

Let `K = 84 × 10^18`, `H = 80 × 10^18`, and let an economic entry be `(r,n,l)` with `n+l>0`. An entry represents either one incoming miner's actual carried receipt or a completed/open ordinary composition period. The exact rational specification gives:

`E_N = Σ 80 r n / [84(n+l)]`, and `E_L = 80T/84 - E_N`.

The candidate's precision policy gives:

`Q_N = Σ floor(H r n/(n+l))`, `Q_L = H T - Q_N`,

`I_N = floor(Q_N/K)`, `I_L = floor(Q_L/K)`.

Receipt entitlements are `perN = floor(I_N/S_N)` and `perL = floor(I_L/S_L)` for positive supplies. The independent Python program uses `Fraction` for the economic oracle and a separately named precision oracle for the candidate rule. It imports neither PoolMath, SDK math nor the edited author models. Agreement with the precision oracle answers implementation-policy questions; disagreement with the ideal oracle quantifies the approximation rather than being silently treated as a model error.

3.2 Recognition and period assignment

Under supported token behavior, define `T = totalAccounted`, `D = totalPaid`, `B = bem.balanceOf(pool)`, and expected recognized reserve `R = T-D`. With a composition, `_sync` rejects `B<R` and recognizes `u = B-R`, making `T'-D=B`. A later identical sync recognizes zero. A real outgoing payment increases D by exactly the decrease in B, leaving `B-(T-D)` unchanged. Consequently previously paid income is not recognized again merely because a user claims. An intentional later donation is new income and is correctly counted again. References: Pool `871-880`, `887-911`.

Admission does the following in one guarded transaction (Pool `623-735`):

1. Validate every incoming ID, current holder, ACTIVE record, collection/ID binding, nonempty burn and cache agreement; reject categories with zero receipt supply.
2. Recognize already-held BEM under the old composition.
3. Claim every registered miner and recognize those actual proceeds under the old composition. A non-REVOKED failure aborts the admission.
4. Checkpoint both category unit amounts; transfer and register the incoming NFTs.
5. For each incoming claim, measure a separate before/after BEM balance difference and split that received amount by that miner's own burn. Add the aggregate carried amount to T and to `accountedBase`, and add the corresponding units to the bases.
6. Add incoming burn to aggregate composition and recognize any still-unassigned balance under the new period.

Because carried income increases both T and `accountedBase`, the next ordinary formula `H × (T-accountedBase)` excludes it. Each carried payment enters total fees once and category bases once. No `pending()` estimate is used. Existing balances are excluded from each incoming claim's

before/after delta. A failing later incoming claim rolls back earlier claims, transfers, registry writes, bases, fees and the first-custody latch along with the transaction.

Attribution assumption: a delta measures everything transferred during that external call, not an economically authenticated source label. The intended Mining implementation must pay only the relevant miner through that claim and the token must have supported balance semantics. A malicious dependency that pays unrelated amounts during the call can defeat source attribution without contradicting the balance-delta calculation. The retained author fork tests characterize examples of the pinned implementation. This review's protocol-fork replay was blocked before execution; the fresh public identity/state reads are not behavioral fork tests.

3.3 The state before any composition

Before the first admission, `N+L=0`; `_sync` returns zero. In reachable genuine states, T, category bases and paid histories remain zero. Preexisting BEM is physically present but not claimable. `reconciliation().unrecognized` exposes it, whereas `unrecognizedBalance()` intentionally returns zero so previews do not allocate it without a composition (Pool `510-516`, `564-570`). These two views have different pre-composition semantics and should not be conflated.

At the first successful admission, carried income is attributed separately. The preexisting balance is then ordinary income under the first admitted batch's aggregate composition. Therefore initial donations or already-paid unregistered-miner rewards can depend on which first composition opens the Pool. This is a stated policy for unattributed balance; it is not covered by the fixed-carried-input permutation theorem. If no valid miner is ever admitted, those pre-composition funds remain inaccessible. That is the retained SALUS #1 unsupported-path limitation.

Unsolicited NFT transfers do not append registry entries and cannot by themselves enlarge the old-miner settlement loop. A third party can, subject to real Mining semantics, claim an unregistered Pool-held miner to the Pool; that new balance follows ordinary recognition, not the unregistered miner's burn. It does not retroactively change already-accrued entitlements. The supported `depositMiners` transaction has no external-transaction gap between transfer and registration. Permissionless Mining claims **before** deposit may instead pay the old holder and reduce what actually travels into the Pool.

3.4 Conservation and the residue bound

At a boundary, the initial zero state satisfies `nandBaseUnits+latchBaseUnits = H × accountedBase`. A checkpoint preserves current units, whose increment splits `H × (T-accountedBase)` into a NAND floor and the exact complementary LATCH remainder. Carried entries likewise sum to `H × carried`. Induction therefore establishes:

```
nandBaseUnits+latchBaseUnits = H × accountedBase,
```

and, whenever composition exists, `nandUnits+latchUnits = H × T`.

With `A=B_fee=floor(T/84)`, `C=floor(2T/84)` and `P=floor(80T/84)`, define:

- Outer residue `r_o = T-A-B_fee-C-P`. Writing `T=84q+r`, `0≤r<84`, gives `r_o=r-floor(2r/84)-floor(80r/84)`, so `0≤r_o≤3`.
- Category residue `r_c=P-I_N-I_L`, which is either 0 or 1 because the category units sum exactly to H T.
- Share remainders `r_N=I_N-S_N perN` and `r_L=I_L-S_L perL`. Each is between 0 and S-1 for a positive supply.

Then:

```
T = A+B_fee+C+S_N perN+S_L perL+residue,

residue = r_o+r_c+r_N+r_L ≤ 4+max(S_N-1,0)+max(S_L-1,0).
```

For a zero-supply category, admission prevents any nonzero burn in that category; its ledger income stays zero. Thus the bound's zero-supply convention is justified for reachable contract states. It is not justified for an arbitrary fabricated PoolMath state that has positive weight and zero shares. **The reported residue bound is sound under these reachable-state conditions and checked-arithmetic assumptions.** It bounds unallocated balance, not lifetime category fairness.

3.5 Paid histories, monotonicity and solvency

Within one period, `floor(dN/W)` is nondecreasing in d. Its complement `d-floor(dN/W)` is also nondecreasing because `0≤N≤W`. Checkpoints preserve attained values; carried additions are nonnegative. Category incomes and per-share values therefore never decrease. The outer fee floors are independently nondecreasing in T.

Each receipt claim checks the caller's current balance equals one, computes `perShare-history[id]`, and sets history to perShare before the external BEM transfer (Pool `827-837`). The category paid counter increases by that difference. Duplicate IDs in an actual batch see the updated history and pay zero on subsequent occurrences. By induction:

`receiptsPaid[N] = Σ claimed[N][id]`, and likewise for LATCH,

```
D = paidA+paidB+paidC+Σ claimed[N]+Σ claimed[L].
```

Every paid bucket is bounded by its accrued bucket. The five unpaid obligations are nonnegative and sum to `T-D-residue`. Under standard, fee-free, non-rebasing token behavior:

```
balance = T-D+unrecognized,
```

so balance covers all unpaid obligations, residue and unrecognized income. Rollback maintains these identities after failed transfers. A negative rebase, confiscation or dishonest balance response is outside this guarantee; the contract fails closed where the deficit is detected rather than socializing it.

Transfers of receipts leave paid history keyed by category/ID. The previous holder loses claim authorization; the new holder inherits only the remaining unpaid entitlement. Approval as an ERC-1155 operator alone does not authorize a BEM claim. Such an operator may transfer the receipt under the holder's granted authority and then claim as its new holder; that is ordinary receipt ownership behavior, not a bypass.

3.6 A/B/C releases and role interactions

`_release` takes one post-sync accrual snapshot, updates only the selected A/B paid counter and C's independent paid counter, and pays both via `_payWithSink` (Pool `843-899`). The other role's paid history is unchanged. At T=42 raw, A=B=0 and C=1; C is still paid. Repeated releases cannot pay C's backlog twice. The destination is the fixed DEAD address; this is an ERC-20 transfer, not a call to burn and not ownership renunciation.

Both outgoing transfers and three net balance checks are atomic. A failed sink transfer rolls back the selected role transfer and all counters. Receipt claims do not call the sink. If a supported token later refused only sink transfers, role releases would fail while receipt claims could remain

available. A/B may overlap at the same address: their counters remain separate. A role migration transfers the unpaid role entitlement to the new authorized role address, while its historical paid counter persists. `setRecipient` is intentionally retained after all owners are zero; it is not a restored setup-owner power. It disallows zero, Pool and sink destinations. It does not authenticate a recipient contract's ability to use received BEM.

3.7 What order independence does and does not establish

For each fixed incoming `(r_i,n_i,l_i)`, the candidate computes the same integer pair, and addition commutes. No intervening ordinary income means later checkpoints add zero. Therefore ordering and transaction partition do not change the summed units, raw category incomes, per-share entitlements or total fees. Only registry order and period metadata differ. This proves the selected SALUS #2 remediation rather than the alternative fixed-round-ratio proposal.

It does not establish equality when a miner's own reward is split between carried and ordinary recognition, when extra ordinary income is recognized across composition boundaries, when pre-composition donations face different first compositions, or when upstream shared state changes actual payments. Recognizing income many times inside **one unchanged period** remains path-independent because the period formula uses cumulative T rather than adding a new split at each sync.

3.8 Finite precision, thresholds and arithmetic limits

For q rounded entries with q≥1, let each NAND flooring error in ledger units be $e_i \in [0,1)$, and $\varepsilon = \Sigma e_i / K < q/K$. For q=0, ε=0. The candidate unit balances represent $E_N - \varepsilon$ and $E_L + \varepsilon$. Hence the correct bounds are:

$$E_N - 1 - q/K < I_N \le E_N,$$

$$E_L - 1 < I_L \le E_L + q/K.$$

These bounds differ from a universal one-raw-unit bound. A tiny representation difference can cross a floor threshold. PC-01 gives both a simple practical-domain raw-threshold example and a valid-integer-domain rational counterexample. The latter's extreme burns have not been demonstrated feasible in the real Mining protocol. The author FFI model actually includes a q/K slack term, while its documentation/messages suggest a strict one-unit guarantee; model agreement cannot validate the stronger prose claim.

Checked multiplication can revert: `participantUnits(r)=r×80×10^18`, followed by multiplication by NAND burn in the split. Full-width `mulDiv` is not used. A sufficient ordinary bound is `delta ≤ floor((2^256-1)/(H×N))` for N>0, together with safe accumulated bases; a more conservative W-based bound is valid but not tight. Incoming per-miner carried income additionally must fit uint120. With uint32 burn and that uint120 cap, a single carried split is well below the uint256 product limit. Lifetime bases and cumulative T still have finite bounds. The exact independent calculations are in `independent-arithmetic.json`.

An instantaneous BEM supply ceiling is not directly a ceiling on cumulative T: payouts can be donated again. A stronger **conditional** economic argument is available if supply is capped at M and the sink never releases or destroys paid C: paid C remains at the sink and unpaid C remains in the solvent Pool, so `floor(T/42)≤M`, giving `T≤42(M+1)-1`. This depends on token/sink behavior, not merely on the Pool source. No realistic overflow was demonstrated.

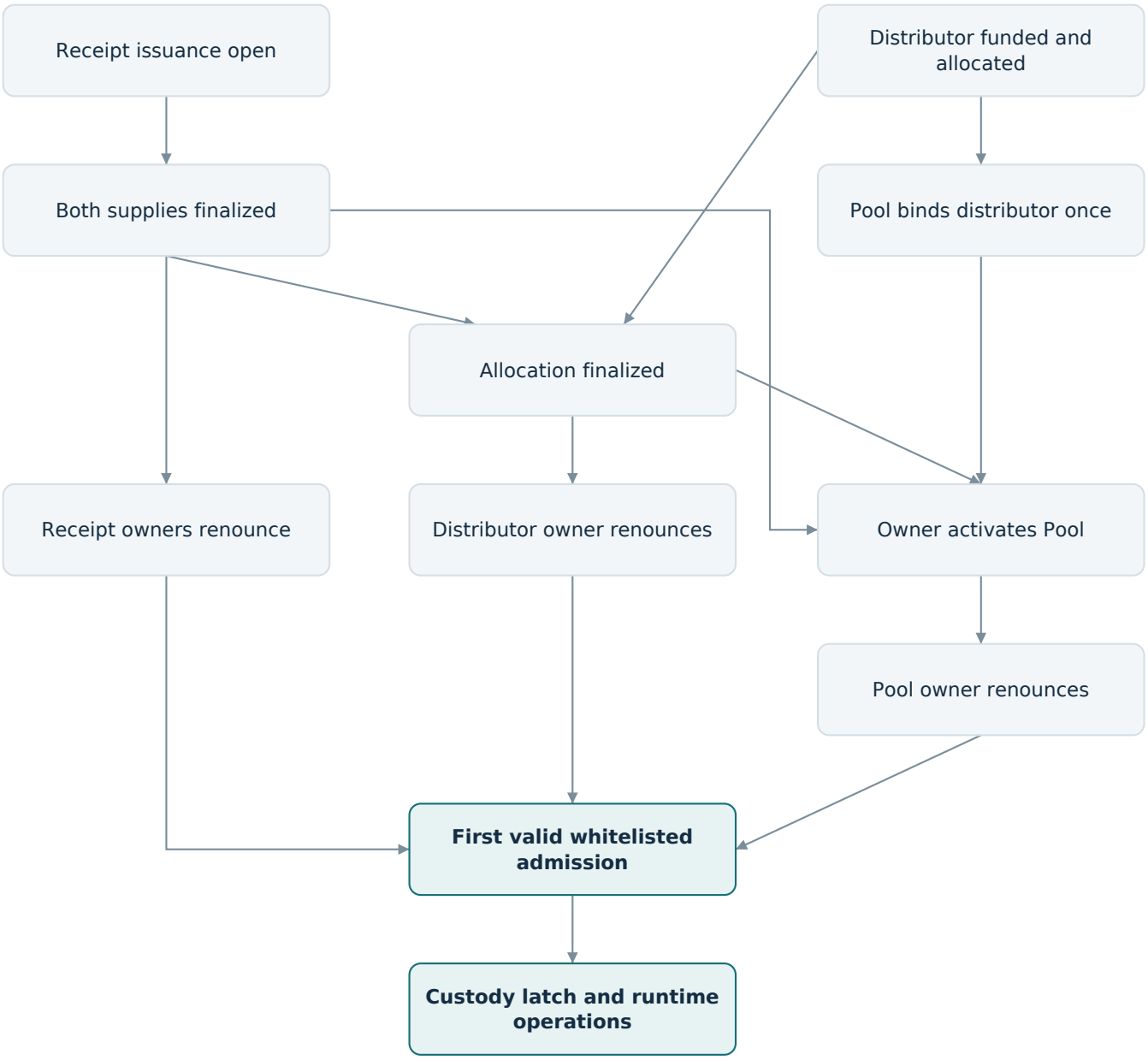
`MinerRecord.circuitId` is uint64, while burns are uint32. ID equality plus permanent registration bounds distinct admitted IDs by 2^64. At most one checkpoint per admission and one carried split per miner gives a nominal rounding-bias upper bound below `2^65/K≈0.4392` raw units even at this infeasible domain maximum. It still does not make the literal one-unit rational-income claim true. `compositionPeriod` is uint256 but stored per miner through an unchecked uint64 conversion; a 2^64-th one-at-a-time admission could truncate that metadata to zero. This is a theoretical unreachable-at-practical-gas scale observation, not a realistic financial exploit. Zero denominators are prevented on the actual admission path; invalid external enum values are rejected by Solidity ABI decoding.

4. Lifecycle, authorization and custody boundary

In this section and the findings, **Pool** means `src/TapeoutMinerPool.sol`, **Receipt** means `src/TapeoutEntitlementReceipt.sol`, and **Distributor** means `src/TapeoutReceiptDistributor.sol`, all at the frozen target. Line references are to those exact files, not a proposed patch.

4.1 State and authority transitions

Figure 1. Setup and custody dependencies for genuine reviewed implementations.



The graph assumes genuine reviewed implementations. It describes formal admission, not the physical possibility of somebody plainly transferring an NFT to the address before setup is complete. Such a transfer does not register a miner or establish a composition.

Table 3. Contract lifecycle and authority

Contract/state	Authority and transition	Irreversible boundary
Receipt, issuance open	Current owner issues sequential IDs, retracts only top IDs held by that same owner, sets URI, transfers ownership, or renounces. Holders/operators can transfer receipts.	Premature renunciation disables issuance finalization permanently.
Receipt, finalized	Mint/burn and issue/retract are disabled. Owner can still set URI and manage ownership until renunciation.	Supply is fixed. Owner-zero also removes URI/setup authority.

Contract/state	Authority and transition	Irreversible boundary
Genuine distributor, allocation open	Eligible genuine receipt holders may fund it. Owner allocates or revokes wallet ranges.	Pool and receipt bindings are immutable. Delivery is not available yet.
Genuine distributor, finalized	Allocation/revocation and new funding are disabled. Beneficiary may claim; anyone may deliver to the already fixed beneficiary.	No owner or public caller can redirect allocation through the reviewed code.
Pool, unbound	Owner binds one distributor reporting the correct Pool/receipts.	The binding cannot be replaced.
Pool, bound but inactive	Owner activates after finalized supplies match and the distributor reports finalized allocation.	Activation and denominators are permanent.
Pool, active before first admission	Admission is still closed unless every one of the four owners reports zero.	Renouncing Pool before activation instead bricks activation.
First valid admission	Whitelisted caller passes all gates; `_openCustody` sets the latch inside the admission transaction.	Any later failure rolls back the latch and every transaction effect.
Runtime	Whitelisted admissions, holder-only receipt income, permissionless harvest/sync/A/B release, holder transfers, and A/B role migrations continue.	No production withdrawal, sweep, upgrade, arbitrary-call or registry-removal path exists.

References: Receipt `58-99,135-150`; Distributor `85-184,276-299`; Pool `293-326,623-752,815-865`.

The necessary setup order is a partial order. Binding can occur before or after allocation finalization. Receipts and distributor can renounce after their own finalization, including before Pool activation. Pool must activate before its owner renounces. The CLI chooses a safer serial procedure; the contracts do not prevent all premature direct renunciations.

4.2 What is enforced, what is operational, and what is assumed

Table 4. Enforcement and trust boundaries

Condition	Enforcement	Assessment
Exact genuine receipt runtime and category	Pool constructor `255-262`	Strong on-chain implementation check for these non-immutable receipt runtimes.
Finalized receipt supplies, matching fixed denominators	Pool activation `313-319`	On-chain, stable in genuine receipts.
Bound distributor reports matching addresses	Pool `296-305`	On-chain view check; does not authenticate implementation.
Distributor reports finalized allocation	Pool `320-321`	On-chain reported state; genuine code must be established independently.
Four owners zero before first admission	Pool `_openCustody`, `738-752`	On-chain; cached after successful first admission.
Pending ownership cannot revive genuine owners	Locked OZ plus absence of initializer/override	Source proof; also fresh local regression.

Condition	Enforcement	Assessment
Entire round's receipt supply covered by reviewed allocation	CLI/SDK full-round verification	Not a universal Pool/Distributor invariant; explicit partial mode exists.
Contribution JSON reflects actual EOA fundraising	Owner/off-chain review	Cannot be proved by these contracts.
Correct BEM/Mining/NFT implementation and future semantics	Deployment choice and external dependency review	Pool only requires code and stores addresses; no automatic upgrade surveillance.
Beneficiary can receive, transfer and later claim from its wallet	Receiver simulation plus operational review	A sample hook/ERC-165 check does not prove every future wallet action.
Safe order of finalization and renunciation	CLI workflow and operator	Direct standard ownership calls remain available.

For genuine contracts, the finality cache is justified. OZ 5.4.0 `Ownable.sol:76-78` calls virtual `_transferOwnership(0)`. `Ownable2Step.sol:52-55` deletes the pending owner before updating ownership; `acceptOwnership:60-65` cannot be used by the old nominee afterward. None of the reviewed implementations exposes an initializer that restores ownership. The fresh regression `testReviewRenunciationClearsPendingOwnerWithoutRestoration` confirms this ordinary execution path. It is not a proof about arbitrary contracts that merely return zero from an `owner()` selector.

4.3 Boundary failures and callbacks

The incoming holder must be the whitelisted caller for `depositMiners`, even if another address granted NFT approval. For retained `registerMiners`, the holder must already be the Pool, and the caller still must be whitelisted. Every incoming key, ACTIVE record, burn sum, category support and cached burn is checked before old settlement begins. NFT transfer and post-transfer `ownerOf` checks occur before successful completion. A failure at any subsequent boundary reverts the whole transaction. A malicious external Mining implementation could execute unexpected callbacks, but value-moving accounting entry points are reentrancy-guarded and actual token deltas are checked; the separate `setRecipient` function instead relies on current-role authorization; the guarantee still assumes honest external holder/payment semantics.

Receipt balances and the holder index are updated before the ERC-1155 acceptance callback. Pool paid history remains keyed by category/ID and is not changed merely by transfer. Distributor delivery updates its cursor and delivered counters before calling the receipt, and reentrancy protection prevents a second delivery. A receiver claiming Pool income after receiving its receipt is the legitimate current holder. If its outer hook later fails, the entire nested transaction rolls back. The genuine distributor itself has no BEM-claim, sweep, arbitrary-call or receipt-approval function, so late beneficiaries retain the accrued unpaid income on their IDs while the distributor holds them. References: Receipt `135-155`; locked OZ `ERC1155.sol:181-198`; Distributor `276-299`; Pool `_claimOne:827-837`.

5. Findings register

The register separates code defects, setup trust, tooling assurance, external assumptions and policy. Related entries are not independent financial losses to be added together. In particular, ST-01 supplies the trust boundary that TL-01/TL-02 fail to close.

Table 5. Categorized findings register

ID	Category	Severity	Finding / disposition
PC-01	Production accounting / precision claims	Low	Same-ratio boundaries can shift one raw unit; universal rational one-unit guarantee is overstated.
PC-02	Production view API	Low	Duplicate IDs inflate raw `claimableBatch` preview; actual claims do not double pay.
PC-03	Production availability	Medium	Complete old-registry settlement creates finite gas capacity and a single-old-miner failure dependency.
PC-04	Production setup availability	Low	SALUS #4: wallet-wide revocation remains unbounded.
PC-05	Production setup operations	Low	SALUS #5: ownership handover can strand retraction authority away from required top IDs.
PC-06	Production setup griefing	Low	SALUS #6: unsolicited eligible receipt inventory can delay finalization and now activation.
ST-01	Setup/deployment authority	Medium, conditional	View-based binding admits a lying owner-zero distributor; genuine identity is an external setup requirement.
ST-02	Setup/full-round policy enforcement	Low	Finalized empty/partial distributor does not establish full-round coverage.
TL-01	Tooling	Medium	Optional distributor field permits `MATCHES` without checking the Pool's actual bound distributor.
TL-02	Tooling	Medium	Independent immutable masking accepts inconsistent copies; getters do not validate operational occurrences.
TL-03	Tooling	Low	`protocol-check` is partial identity verification and omits its own chain check.
C-01	Canary only	Informational	SALUS #7: critical one-time participant binding has no event.
C-02	Canary only	Informational	SALUS #8: recovery/admission token IDs remain duplicated literals.
EX-01	External dependency	Material dependency, current authority qualified	Fresh beacon owner-zero supersedes historical active-owner premise; complete collection/protocol assurance remains bounded.
EX-02	External economic behavior	Medium, conditional	Upstream supply cap can change actual carry inputs by order at the same timestamp.

ID	Category	Severity	Finding / disposition
EX-03	External compiler dependency	Low upstream severity; unresolved release gate	SOL-2026-7 includes the pinned compiler/settings; candidate-specific miscompilation is unverified.
POL-01 -05	Intended policies	Acceptance required	Historical burn weights, fixed whitelist, unsupported transfer recovery, irreversible setup/recipient mistakes and off-chain allocation truth.

6. Production-contract findings

PC-01 - Precision is much better, but it is not exact rational or recognition-boundary invariance

Severity: Low. **Confidence:** High. **Evidence:** `SOURCE_CONFIRMED` and `VERIFIED_BY_EXECUTION` through independent Python and fresh Solidity regression tests. **Actor/preconditions:** Ordinary legitimate recognition separated by an admission, or differently segmented carried/ordinary income; a whitelisted depositor can choose timing. No malicious receipt holder privilege is needed for the arithmetic effect.

Violated assertion: The documentation implies elimination of same-ratio checkpoint drift and a universal one-raw-unit distance from exact rational category income. The actual ledger preserves the sum of units, not each category's exact rational allocation across arbitrary boundaries.

References: `src/PoolMath.sol:98-118`; Pool `_checkpoint:791-798`; `docs/ACCOUNTING.md:159-165`; `docs/SALUS_AUDIT_REMEDIATION_REPORT.md:189-200`; `test/salus-ffi/model.py:250-260`; `test/audit/model.py` rational-bound changes in the baseline diff.

Minimal reproduction, all amounts raw BEM and one receipt per category:

1. Admit a zero-carry miner burning 1 NAND and 2 LATCH.
2. Recognize 1 raw BEM.
3. Admit another zero-carry 1:2 miner, creating a checkpoint with unchanged ratio.
4. Recognize 62 raw BEM.

Table 6. Raw-unit threshold comparison

History at T=63	NAND ledger units	NAND income	LATCH income	C	Residue
One ordinary period at 1:2	1,680,000,000,000,000,000,000	20	40	1	2
Ordinary entries 1 then 62, same ratio	1,679,999,999,999,999,999,999	19	40	1	3

The difference is one ledger unit, which crosses an integer entitlement threshold. A real `pool.claim(NAND, 1)` in the second regression pays 19 raw units, not 20. Splitting one miner's reward as carry 1 plus ordinary 62 has the same threshold effect compared with carry 63. Neither example refutes permutation invariance for an unchanged vector of per-miner carried inputs.

A separate integer-domain counterexample has two miners with 1 raw carry each:

- X burns `(1,278,728,898, 3,016,238,393)`.
- Y burns `(3,230,986,741, 1,063,980,528)`.

All fields fit uint32. Exact ideal NAND income is `387381622661682563860 / 387381622661682563859`, strictly greater than 1. The candidate produces NAND units `K-1`, hence raw NAND income 0. Its shortfall is strictly greater than 1 raw unit. The difference beyond 1 is minuscule, and these extreme burns have not been shown attainable in the deployed Mining protocol. This establishes a false universal contract-domain assertion, not a demonstrated economically useful real-protocol exploit.

Impact: Small category threshold differences and inaccurate accounting guarantees. Per-receipt flooring can amplify the difference in presently claimable obligations: with S_N=20 and S_L=1, the same example changes NAND per-ID income from 1 raw to 0 for all 20 IDs, so aggregate NAND receipt obligations are 20 raw versus 0 and residue is 2 versus 22 (still below the bound 23). This is deferred dust/payout timing, not a 20-raw loss from the category ledger; further income can release the held residue. That propagation example was independently executed in Python (`receipt-threshold.json`), not added to the Solidity test count. Conservation, balance solvency and paid-history monotonicity remain intact. Residue bounds do not prove individual rational fairness. At BEM's observed 8 decimals, one raw unit is 0.00000001 BEM.

Remediation: State the exact q/K precision bound derived in section 3.8; qualify recognition-boundary invariance; add the two explicit threshold regressions. If exact no-boundary-effect economics is mandatory, commission a separate accounting design review of persistent rational/remainder accounting and its denominator changes. Do not add an arbitrary epsilon or round both categories up, which could violate conservation. The minimal safe correction is to document and explicitly accept the implemented precision policy.

Regression: Preserve `testSameRatioCheckpointChangesRawThreshold`, the carry/ordinary variant and `testRationalDeficitCanExceedOneRawUnit` in the separate review proof sources; test conservation and monotonicity alongside these counterexamples. The author's current FFI tests allow a q/K slack even though their messages describe a strict one-unit bound. Passing them does not prove the stronger prose claim.

PC-02 - Raw batch previews count duplicate IDs more than once

Severity: Low. **Confidence:** High. **Evidence:** `SOURCE_CONFIRMED`; `VERIFIED_BY_EXECUTION` by `testDuplicateClaimPreviewIsNotExecutableSum`. **Actor/preconditions:** A consumer supplies duplicate IDs directly to the raw Pool view. **Invariant:** A quoted batch claim should reflect its executable economic payment for the same input.

References: Pool `claimableBatch:535-550`, `_claimMany:815-824`, `_claimOne:827-837`; SDK `canonicalClaimIds:169-184` and claim-plan validation `188-197`; CLI `590-605`.

With one NAND ID owed 100 raw BEM units, the raw preview for `[id,id]` reports 200 raw units. The state-changing batch pays 100 raw units: the first iteration advances history and the second owes zero. No insolvency or double claim is possible through this discrepancy. Official SDK/CLI normalization mitigates its own user path, but direct integrators can overstate expected proceeds or make a poor gas/transaction decision.

Remediation: Deduplicate or reject repeated IDs in the contract view, or specify set semantics clearly and expose an equally strict public preview. Retain the actual payment logic. **Regression:** Preview and execute repeated and mixed-category IDs; NAND ID1 and LATCH ID1 are separate

entitlements and must not be merged across categories. This is retained behavior, not introduced by the 2.2.0 ledger.

PC-03 - Every admission depends on settling the complete old registry

Severity: Medium availability/design limitation. **Confidence:** High for the dependency; current reachability of a hostile genuine non-REVOKED failure is not established. **Evidence:** `SOURCE_CONFIRMED`, with author capacity evidence separated below and local fault tests identified in section 12. **Actor/preconditions:** Legitimate registry growth, or one old miner whose claim persistently fails without status REVOKED. No attacker is required for growth exhaustion.

Invariant at risk: Later authorized miners made from leftover materials can continue joining within feasible chain transaction limits. **References:** Pool `_admit:658-663`, `_settleOld:780-788`, permanent `_minerKeys`, `MAX_ADMISSION_BATCH`; `sdk/gas.ts:9-41`; `test/audit/AuditAdmissionLiveness.t.sol:21-61`.

Each admission claims every old miner before checkpointing. A failure is ignored only after an external status read confirms REVOKED. This strict behavior protects old-period attribution but couples all future admissions to every existing key. Incoming batch size 50 does not bound old-registry work. Splitting a new batch into smaller transactions repeats the same old work. Registry entries cannot be removed, including revoked ones.

A precise local fault case is: admit A; have the mock reject A's claim while reporting ACTIVE/COOLING; try admitting B. `SettlementFailed` reverts B's transfer and all earlier settlements. The mock's COOLING state is fault injection, not proof that a public attacker can put a current genuine Pool-held miner into COOLING. Current external lifecycle reasoning reportedly rules that route out.

Impact: New admission can become unavailable. Permissionless paginated harvest isolates per-key claim failures, and existing BEM claims/releases remain possible if their own dependencies function. This is not inherently loss of previously recognized BEM.

Table 7. Admission capacity evidence

Evidence type	Quantity	What it establishes
AUTHOR_RECORDED real Mining fork	24 miners; with 23 old, next admission measured 877,605 internal gas; largest sampled marginal 27,857	A small real-protocol sample, excluding standalone transaction base gas.
AUTHOR_RECORDED first admission	514,781 internal; 538,382 actual local-fork transaction gas	A particular first-deposit path, not every constructor/record branch.
VERIFIED_BY_EXECUTION fresh mocks, matching retained measurements	400 old + 1 new: 9,035,212; 401 old + 10 new: 10,484,860	Mock capacity only; `fresh-gas-admission-local.json`.
SOURCE_CONFIRMED CLI model	`300,000 + 30,000*old + 250,000*incoming`, plus 150,000 on first admission	Conservative planning formula, not an on-chain guarantee.
Model/extrapolation	401 old for a one-miner plan under 3/4 budget; roughly 590 near full cap by extrapolation	Neither figure is a measured real-protocol maximum.

The protocol-level single-transaction cap of 16,777,216 gas is independently supported by BNB Chain's current BEP-652 and Mendel activation announcement, which states mainnet activation on

28 April 2026. This is separate from the block gas limit. The requested fresh protocol-fork replay was blocked before execution, as recorded in section 12; the 24-miner real-protocol sample remains AUTHOR_RECORDED.

Remediation: Establish and enforce a launch-specific maximum registry with measured spare capacity for future materials; test real protocol branches, revoked keys and the actual transaction/Safe envelope. If the required size exceeds the bound, design staged settlement with a composition lock/snapshot and formally review its economics. Simply skipping unsettled ACTIVE miners would invalidate period attribution.

Regression: Persistent non-REVOKED failure must fail atomically; REVOKED handling must remain precise; measure the planned maximum and one additional admission on a pinned real-protocol fork. Include calldata/base/overhead, not only gasleft differences.

PC-04 - SALUS #4: revocation still loops through an unbounded wallet allocation

Severity: Low, setup availability. **Confidence:** High. **Evidence:** `SOURCE_CONFIRMED`; an exact production maximum was not newly measured. **Actor/preconditions:** Owner imports a large/fragmented allocation across many calls and later needs to correct that beneficiary. Public users cannot allocate ranges to inflate it.

Invariant: Any accepted pre-finalization allocation should have a feasible correction path. **References:** Distributor `allocate:123-148`, `revokeWallet:151-168`; `sdk/gas.ts:192-205`; CLI `398-403`; `docs/OPERATIONS.md:195-198`.

`allocate` can append ranges over arbitrarily many transactions. `revokeWallet` then visits every range and every ID and deletes them atomically. With sufficiently many slots, correction exceeds the per-transaction cap. A CLI gas refusal prevents a wasteful attempt; it does not provide bounded recovery. The narrative's approximate 1,550-ID threshold is an estimate, not a proven universal boundary.

Impact: Setup must retain the incorrect allocation, be abandoned/redeployed, or remain incomplete. The new Pool activation gate makes incomplete allocation delay custody start. It does not create post-finality revocation power.

Remediation: Add bounded/cursor-based correction with exact safe counters and finalization blocked while correction is incomplete, or enforce a conservative per-wallet ID/range cap before acceptance. **Regression:** Import a fragmented wallet over several transactions beyond one-call capacity; prove bounded revocation progresses without double decrement, reallocation is correct, and finalization cannot race incomplete correction.

PC-05 - SALUS #5: ownership transfer separates retract authority from held inventory

Severity: Low, setup operations. **Confidence:** High. **Evidence:** `SOURCE_CONFIRMED` and fresh Solidity execution. **Actor/preconditions:** A holds the highest issued receipt IDs, then B accepts ownership before over-issuance is corrected. **Invariant at risk:** The current issuer can correct top-ID issuance without depending on an unavailable previous holder.

References: Receipt `retract:72-85`; locked OZ `Ownable2Step.sol:43-65`; Pool activation `313-319`. The fresh test is `testReviewOwnershipRotationSeparatesRetractAuthorityFromInventory`.

A issues IDs1-2 to A and hands ownership to B. B cannot retract ID2 because B does not hold it; A cannot retract because A is no longer owner. A cooperating transfer of ID2 to B restores the correction path. Therefore this is **not automatically a permanent lock** after every handover. If A's key is lost or A refuses, correction may be impossible.

Impact: Supply/round correction can stall before permanent finalization. **Remediation:** Finish issuance review and correction before handover; include required top-ID transfers in the handover procedure. A code redesign must preserve holder consent rather than silently giving the new owner seizure power. **Regression:** Handover with and without inventory transfer; attempted third-party retraction must fail; successful correction must precede finalization/renunciation.

PC-06 - SALUS #6: unsolicited eligible receipt funding delays finalization and activation

Severity: Low, setup griefing. **Confidence:** High. **Evidence:** `SOURCE_CONFIRMED` and fresh Solidity execution. **Actor/preconditions:** An outside holder owns a genuine, issued, still-unpaid receipt not yet in inventory while allocation remains open. Counterfeit contracts and zero-value transfers do not suffice. If the entire finalized supply is already in inventory, no such outside receipt exists.

Invariant at risk: A reviewed complete allocation remains ready to finalize between preflight and execution. **References:** Distributor hooks `85-113`, `_acceptOne:260-269`, `finalizeAllocation:170-184`; Pool `activate:320-321`.

Assign ID1 already in inventory. An outside holder transfers eligible ID2 before finalization. Inventory rises to two while allocated remains one. Finalization reverts `InventoryNotCovered`; activation also reverts because allocation is unfinalized. Existing beneficiary assignments are not changed. The fresh test `testReviewUnsolicitedFreshInventoryBlocksFinalizationAndActivation` executes both failures.

Impact: Finite eligible outside inventory can force replanning, review and gas costs; the new gate extends delay to custody opening. It does not allow an attacker to rewrite a finalized allocation or seize income after finalization.

Remediation: Separate funding closure from allocation finalization, restrict funding sources where policy permits, or establish full finalized-supply inventory before publishing the final transaction sequence. **Regression:** Insert genuine eligible funding between preflight and finalization, prove no silent allocation change, permit reviewed assignment of the new ID, and reject all funding after finalization.

7. Setup, deployment and tooling findings

ST-01 - Reported zero ownership is not genuine distributor identity

Severity: Medium conditional setup risk. **Confidence:** High. **Evidence:** `SOURCE_CONFIRMED` and `VERIFIED_BY_EXECUTION` on the candidate with a local dishonest distributor. **Actor/preconditions:** Malicious/compromised setup owner, or operator persuaded to bind unreviewed code. Binding is owner-only and one-time; an arbitrary runtime caller cannot replace a genuine bound distributor.

Invariant at risk: First admission establishes permanent beneficiary allocation in the genuine reviewed implementation. **References:** Pool `bindDistributor:293-305`, `activate:311-326`, `_openCustody:738-752`; `src/interfaces/IReceiptDistributor.sol:6-15`; `test/salus/SalusFinality.t.sol` FakeDistributor fixture; remediation report `323-332`.

The local reproduction binds a contract reporting the right addresses, finalized allocation and owner zero, but retaining a separate administrator. Genuine receipts and Pool are finalized/renounced. Admission succeeds. The hidden administrator subsequently moves the receipt to a chosen wallet, changes its reported finalized flag, and another admission still succeeds because finality was cached. This is executed by `testReviewLyingZeroOwnerDistributorRetainsPowerAfterAdmission`.

Impact: The state gate can give false assurance about irreversible allocation. The setup owner already chooses issuance and beneficiaries, so this finding does not create a new unprivileged power after an honest genuine deployment. It demonstrates why deployment authenticity is a required security condition and why TL-01/TL-02 matter.

Remediation: Authenticate full runtime using consistently patched constructor immutables and verified deployment provenance; consider a reviewed factory or suitable on-chain identity mechanism. Constructor immutables prevent naive comparison to a zero-filled template, not identity verification in general. A real genuine distributor's finalized allocation and owner-zero state are stable; repeated view polling cannot substitute for that fact.

Regression: Accept a genuine expected deployment; reject the ordinary FakeDistributor, an upgradeable look-alike, and a runtime with operational immutable occurrences inconsistent with checked getters. Then verify no genuine owner or allocation restoration after custody begins.

ST-02 - Empty or partial finalization can still open formal custody

Severity: Low when the full-round CLI is mandatory; potentially material setup error if that assumption is lost. **Confidence:** High. **Evidence:** `SOURCE_CONFIRMED` and fresh Solidity execution. **Actor/preconditions:** Setup owner leaves some/all issued receipts outside the genuine distributor. **Invariant:** Full-round production readiness includes all issued receipts and the reviewed beneficiary record.

References: Distributor `finalizeAllocation:170-184`; Pool `activate:311-326`; CLI `finalize-allocation:406-423` and `activate:447-460`; SDK supply coverage in `verify.ts`.

The contract compares allocated inventory to accepted inventory, not inventory to total receipt supply. The fresh test issues the sole NAND ID to Alice, leaves the genuine distributor empty, finalizes it, binds/activates/renounces everything, and admits a NAND miner carrying 105 BEM.

Custody opens and Alice receives 100 BEM. No denominator disappears: outside IDs remain entitled.

Impact: Finality is not authentication of a complete contribution round. Explicit partial mode can be intentional, but it must not masquerade as the specified full round. **Remediation:** Require a pinned full-supply artifact verification before renunciation, with recorded inventory provenance and reviewed JSON hash. If full coverage is a hard on-chain requirement, add it in a separately reviewed revision while counting already delivered receipts correctly. **Regression:** Empty/partial coverage fails full-round readiness; explicit partial status stays visibly qualified; delivered inventory retains valid coverage.

TL-01 - Omitting the distributor produces an unqualified deployment match

Severity: Medium. **Confidence:** High. **Evidence:** `VERIFIED_BY_EXECUTION` against the actual frozen SDK with an in-memory read-only RPC fixture; related Pool admission behavior is separately executed in ST-01. **Actor/preconditions:** An incomplete addresses manifest, including a deliberate omission, is used with an active/bound Pool. **Invariant:** A positive deployment identity result covers every contract controlling the custody/receipt allocation boundary.

References: `sdk/config.ts:32-38` marks `distributor` optional; `sdk/deployment.ts:93-116` checks the distributor and reads `pool.distributor()` only inside `if (addresses.distributor)`; CLI write preflight `283-293` and admission branch `499-531`.

The fresh `review-proof/identity-review.ts` fixture supplies matching receipt/Pool templates and receipt-address getters but no distributor manifest field. `checkDeployment` returns `MATCHES`, lists only NAND/LATCH/Pool as checked, and performs **no call to** `pool.distributor()` **and no code query for its bound distributor**. Including the same unreviewed four-byte distributor in the manifest correctly changes the result to `MISMATCH`, demonstrating that the missing branch is the defect. The schema accepts the incomplete file.

`deposit` requires the Pool address and may obtain finality from reported views; it does not require the manifest's distributor. Bind/activate commands themselves do require a distributor, so an ordinary complete-manifest setup flow is better protected. The claim is not that every CLI workflow automatically binds a fake; it is that the advertised universal checker result is false assurance for a supported admission input shape.

Impact: A central operational mitigation for ST-01 can be bypassed by omission. **Remediation:** For bound/active/custody states, discover the distributor from the Pool and require complete code/address verification whether or not the caller supplied it. Before binding, return a clearly scoped incomplete/prebinding result rather than unqualified deployment `MATCHES`. Require a complete immutable launch manifest for custody readiness.

Regression: Empty distributor field on a bound Pool must cause discovery and verification or a fail-closed incomplete result; wrong discovered code/bindings must reject; a genuine full deployment must pass. Keep normal prebinding receipt-only planning explicitly separate.

TL-02 - Masked immutable occurrences can disagree while getters remain correct

Severity: Medium. **Confidence:** High for the verification defect. **Evidence:** `VERIFIED_BY_EXECUTION` for the pure SDK masking probe and fresh artifact comparison; `SOURCE_CONFIRMED` for compiled call-site/getter reasoning. A deployed operational malicious-runtime EVM demonstration was **not executed** and is not claimed.

Actor/preconditions: An unreviewed runtime is supplied for deployment verification, e.g. code installed through a constructor other than the reviewed creation bytecode. Arbitrary runtime callers cannot mutate the genuine immutable code. **Invariant:** Accepted runtime semantics use the same constructor addresses consistently as the verified public getters and intended system.

References: `sdk/deployment.ts:17-28,56-114`; `scripts/export-abi.mjs:23`; `sdk/bytecode.json` immutable references; Distributor immutable declarations and `_receipt` helper; fresh `immutable-build-review.json` and `IMMUTABLE_IDENTITY_REVIEW.md`.

`compareRuntime` masks each compiler-reported immutable slot independently. The exporter flattens `immutableReferences` and loses each AST declaration's grouping. The reviewer set a different value in each of the distributor's eleven masked occurrences; the actual `compareRuntime` accepted the result. It correctly still rejects length/non-masked differences. This probe is a defensive byte-comparison test, not a full deployed exploit.

Fresh compiler output exactly reproduces the frozen template and groups those eleven occurrences into three variables:

Table 8. Distributor immutable occurrences

Immutable	Byte offsets of 32-byte occurrences	Checked public getter uses	Separate operational occurrence
`pool`	1956, 5488, 7352	5488	Other sites remain masked
`nandReceipt`	2727, 4921, 7180, 7556	4921	`_receipt` uses 7556
`latchReceipt`	2976, 5419, 7221, 7592	5419	`_receipt` uses 7592

The fresh artifact's AST IDs differ from a possible future compilation; names/group membership, not incidental AST numeric IDs, should be preserved. Static dispatcher tracing shows the checked getters return different in-code copies from the helper's copies. Therefore honest getter responses cannot establish all operational bindings. The Pool itself has repeated immutable occurrences and is subject to the same verification-design issue.

Impact: A `MATCHES` result establishes opcode/template shape outside the masks but not semantic identity of all masked constants. Runtime address divergence could cause calls to the wrong dependency or break delivery/accounting. No such altered production deployment was observed. The genuine constructor sets all copies consistently; the defect lies in accepting other runtimes as equivalent.

Remediation: Preserve compiler immutable grouping and expected semantic names/values, validate every occurrence and width, and compare the fully instantiated runtime exactly. Check complete expected bindings even when no optional system file is supplied. Deployment provenance that verifies the exact reviewed creation transaction is valuable additional evidence, not a reason to leave an incomplete runtime checker labeled definitive.

Regression: For every immutable, change each non-getter occurrence independently while leaving getters correct; reject every inconsistency. Correctly instantiated genuine runtimes across supported constructor values must pass. Also reject unknown/missing groups and non-immutable mutations. A separately authorized local operational test can further demonstrate impact; the acceptance defect is already established without it.

TL-03 - `protocol-check` omits network and several relevant identities/authorities

Severity: Low. **Confidence:** High. **Evidence:** `SOURCE_CONFIRMED`; fresh dependency snapshots independently illustrate why state needs a block/time. **Actor/preconditions:** Operator uses the wrong endpoint or interprets `IDENTITY_MATCHES_RECORDED` as a complete dependency attestation. **Invariant:** The release dependency check should cover the network, runtime identities and authority assumptions on which custody relies.

References: `cli/pool.ts:252-272`, before normal address/chain setup at `275-276`; `sdk/constants.ts:7-21`.

The command checks BEM decimals, Mining implementation address and hash, NFT beacon address and implementation address. It does not fetch chain ID or compare BEM code, Mining proxy code, linked-library code, NFT proxy/beacon/implementation code hashes, Mining owner, beacon owner or pause/initialization state. Normal later write paths do enforce chain binding; this finding does not erase that protection.

Impact: Partial evidence can be mistaken for full release readiness. The fresh change from a historical nonzero beacon owner to current zero would not be expressed by this status. **Remediation:** Validate chain56 and the complete release-specific dependency tuple at a pinned block, report each authority/state observation and mark omissions explicitly. Avoid silently hardcoding BSC defaults when an intended custom protocol configuration differs. **Regression:** A read-only fake endpoint with matching currently checked fields but wrong chain/code/authority must fail the complete attestation or return an explicitly partial result.

8. SALUS #1-#8 reassessment

The PDF's findings table marks all eight issues **Pending**. Its appendix identifies `c3cb93375737de7ec63baa9f390edd0c15a5c7a4`, and its six file hashes independently match that revision. Adding the PDF at `c78a6d1` did not make that commit the audited source. SALUS examined the older 102-part model, before the 2.1.0 C/sink and 2.2.0 ledger changes. These recommendations are this review's conclusions; only SALUS can change the status in its own report.

Table 9. SALUS findings reassessment

SALUS	Original severity	Current independent assessment	Recommendation / remaining limit
#1: pre-registration revocation lock	Medium	The original unsupported path still exists: plainly transferred miners can be revoked before registration, and with no eligible miner left the Pool can retain BEM without any composition/recognized entitlement. `depositMiners` avoids the transfer-registration gap by validating, transferring, registering and accounting atomically. Unsolicited NFTs do not automatically enter the registry.	**Supported-path mitigation plus explicit policy acceptance, not code-level repair of the original lock.** Retain the unsupported-path warning, prove atomic failures, and obtain SALUS acknowledgment of the elected scope. Do not label all direct transfers safe or recovered.

SALUS	Original severity	Current independent assessment	Recommendation / remaining limit
#2: incoming order changes category allocation	Medium	Per-miner actual carry uses each miner's own burn and commutative integer units. The required four orders/batches and future aggregate composition are independently executed. No carry leaks back into ordinary `T-accountedBase`.	**Substantively remediated for identical realized per-miner inputs.** Qualify timestamp/upstream-cap assumptions; correct the precision overstatement in PC-01. This is per-miner carry, not adoption of SALUS's alternative fixed round ratio.
#3: centralization risk	Medium	Genuine receipts/distributor finalize and all four genuine owners must be zero before formal custody. Pending Ownable2Step transfers cannot restore ownership. A/B role migration and whitelisted admission intentionally remain. Setup owner, full-round truth and distributor authenticity are still trusted boundaries.	**Partially/conditionally addressed.** Genuine-code post-setup authority is materially reduced. ST-01, TL-01/TL-02 and irreversible setup checks prevent a broad unconditional closure recommendation. Owner-zero is not a generic contract-authenticity proof.
#4: unbounded wallet revocation	Low	Unchanged unbounded range/ID loop can make correction infeasible. CLI estimates/refusal do not repair it.	**Open**, PC-04. Bound accepted allocation or implement reviewed incremental correction; test real largest beneficiary and fragmentation.
#5: handover and over-issuance correction	Low	New owner may lack the top IDs needed for `retract`; old holder no longer has owner authority. Cooperative transfer can recover, so the lock is conditional.	**Open**, PC-05. Explicit correction/handover order and inventory transfer; preserve holder consent.
#6: unsolicited receipt funding	Low	A holder of eligible outside inventory can increase inventory before finalization. Existing allocation is unchanged but finalization fails; new Pool activation also waits.	**Open**, PC-06. Funding closure or full inventory discipline, with the exact outside-inventory precondition stated.
#7: missing critical-state events	Informational	`CanaryParticipant.bindPool` still writes binding without an event. Production Pool does emit `DistributorBound`/`CustodyOpened`; they are different functions/contracts.	**Open, canary only**, C-01. No production economic defect implied.
#8: magic token IDs	Informational	Canary admission and recovery still contain literal IDs3204/12485; the generator patches both.	**Open, canary only**, C-02. Centralize and test reviewed constants in any future canary revision.

Existing remediation labels, assertion files and prior internal reports were treated as leads. Adapted reference models were compared with the specification rather than accepted as independent proof. In particular, changing a model to mirror NAND flooring and LATCH remainder validates agreement with that representation, not a universal ideal-rational fairness promise.

9. Integration, generated artifacts and separate canary scope

9.1 ABI and consumer migration

Table 10. Integration changes

Changed surface	Independent assessment
`VERSION` and package	Source reports `TapeoutMinerPool/2.2.0`; package is `2.2.0-rc.1`. The guarded SDK summary reader accepts this production read layout, with explicit local-only canary opt-in.
`state()` replaced by `ledgerState()`	Pool `485-497` deliberately removes the old selector. A stale 2.1.0 state caller fails rather than interpreting a new state tuple under old names.
Base getters/meaning	Old public whole-raw `participantBase`, `nandBase`, `latchBase` getters are removed. New private accounted/category bases are read through `ledgerState`; NAND/LATCH bases are ledger units, not raw BEM.
Cumulative accrual/reconciliation	Their 2.1.0 tuple shapes remain, including C. Economic values change under carried attribution/fractions. Pre-2.1.0 decoders can silently shift fields if they bypass version checks.
Miner record	Adds uint120 actual carried income. Existing stored-layout assumptions, direct struct decoders and indexers need the current ABI.
Events	`CompositionCheckpoint` reports totalAccounted and ledger-unit bases; `CarriedIncomeAttributed` and admission fields distinguish actual carry and ordinary/new-period income. An event signature can retain integer types while its units/meaning change, so event consumers need version/deployment binding.
Binding/finality commands	One-time binding, activation after allocation finality, four renunciations, and first-admission allowance are reflected in CLI/SDK. TL-01/TL-02 remain assurance gaps.

Source: Pool `66-74,98-104,143-152,205-239,485-497`; SDK `pool.ts:312-413,415-478,515 onward`; CLI setup/admission branches. This is an unreleased new deployment candidate, not a reviewed in-place storage upgrade or migration of a live production Pool.

Fresh SDK execution confirmed known-version checks, refusal of missing/unreadable/future/legacy versions before sensitive tuples are read, and refusal of canary layout unless the local policy is explicitly enabled. The loopback CLI tests exercised a real 2.2.0 Pool, local canary-3 and a legacy version stub. A separate audit test showed an old decoder can return plausible but shifted numbers without throwing. This is why the read guard is necessary. `VERSION()` alone authenticates layout convention, not genuine code: a lying contract can return any string. The guard is not embedded in every exported low-level helper: `readMiners` (`sdk/pool.ts:515-533`) directly decodes `miner()` without first calling `assertReadLayout`; `readFinality` and history helpers likewise rely on caller context. Direct SDK consumers should call the guard and pass a pinned block. The CLI inspect/reconcile paths do pin a block and use the guarded summary. When `readPoolSummary` is called without a block number, its separate latest reads are not themselves an atomic snapshot; admission preflight may become stale, although the transaction still re-enforces contract gates.

The earlier operational fixes remain visible in source and the 78 fresh SDK tests: chain binding and offline chain labels, full-round supply coverage and allocation JSON/provenance checks, set-based

claim planning, bounded Safe batches, conservative delivery planning after another caller advances a cursor, receiver compatibility checks, and structured negative-test evidence. Canary negative-test infrastructure distinguishes expected decoded errors, wrong errors and unverified transport/metadata outcomes. The older text-search matcher in a retained audit fixture is historical comparison code; it is not the current workflow verifier.

9.2 Fresh build and artifact identity

VERIFIED_BY_EXECUTION: Source-only compilation with solc0.8.24, optimizer200, viaIR and Paris reproduced the checked-in production creation bytes and runtime templates exactly. An in-memory execution of the export formatting logic reproduced whole `sdk/abi.ts` and `sdk/bytecode.json`, including all three production and three mock entries. No generated file was overwritten.

Table 11. Generated artifact identities

Artifact	Result / SHA-256
`sdk/abi.ts`	Exact fresh export match; `d98fe44cd81f414804504b32ba0816aa241adf91578a7865d22c9832f32daef2`
`sdk/bytecode.json`	Exact fresh export match; `8e04aa24409121ac3ed6257d68a0ee31630a618348d457c01625af96572f713b`

Table 12. Runtime size measurements

Implementation	Fresh runtime bytes	Headroom against the frozen 24,576-byte EIP-170 build gate
TapeoutMinerPool	23,168	1,408
TapeoutEntitlementReceipt	8,743	15,833
TapeoutReceiptDistributor	7,680	16,896
RecoverableTapeoutMinerPool	24,439	137
CanaryParticipant	3,410	21,166

The threshold is the conservative limit used by the frozen build/release procedure and specified by EIP-170. The official BSC source also contains future fork-specific limits; this review does not assume a larger future limit is active or available to rescue oversized builds. Every future fix/compiler/dependency change must rerun the chosen deployment-chain size and initcode checks. The production Pool's creation bytes include constructor logic; runtime size alone is not a complete transaction gas test.

Artifacts matching proves build linkage, not all semantics. TL-02 is present even though the generated files are current. Receipt/distributor logic being unchanged likewise does not make their metadata hashes interchangeable across imported-source/build settings.

9.3 Canary-3 separation

The source and generator diff were reviewed, and the exact-generation `--check` passed within the fresh SDK tests. The generator pins production Pool SHA-256 `f8b8d4d0014bb20fb5d81262854ad4be6a26b576d6b50e002d07e4ffa7d56df1`. It adds the canary version/name, recovery controller, single depositor equal to owner, closure gates and token allowlist/return path. It does not silently insert recovery into production.

The canary's immutable `recoveryController` remains powerful after ordinary owner renunciation. It can shut the canary permanently, then return only the permitted NFTs to itself. `emergencyReturn` performs collection ownership/transfer checks, but no Mining or BEM call. A hostile collection can block even that escape. Closure disables admissions and harvest; for an already activated canary, accounting synchronization, claims and releases remain available. If shutdown occurs before activation, activation is also closed and those functions still fail `NotActive`. Source: `src/canary/RecoverableTapeoutMinerPool.sol:638-657,686`; generator `scripts/canary/build-source.mjs`. These are intended canary powers, not production behavior.

C-01 / SALUS #7 - missing bind event. Informational; high confidence; `SOURCE_CONFIRMED`. Controller-authorized `src/canary/CanaryParticipant.sol:40-49` changes the one-time Pool binding without emitting an event. The assurance affected is event-based configuration monitoring, not authorization or fund accounting. State polling/transaction inspection works today. In a future canary revision, emit `PoolBound(candidate)` and test exactly one correctly indexed event on successful binding and none on failed rebinding.

C-02 / SALUS #8 - duplicated token literals. Informational; high confidence; `SOURCE_CONFIRMED`. Recovery at canary line649 and admission at686 independently name IDs3204/12485; generator lines44/61 patch those sites. The actor/precondition is a future maintainer changing only one occurrence. No current mismatch was found. The invariant is a common reviewed admission/recovery allowlist. Centralize constants in a future revision and test both accepted IDs plus rejection of neighboring/unlisted IDs.

Historical canary-1 did use mainnet with controlled test assets and the older 102-part model. Its approvals were pinned to the old candidate, including the retained `c4bc022` reference, not this release. Canary-2/3 local evidence, historical partial automation and recovery records were not treated as authorization for new live activity. This review did not run live canary, preparation, approval or recovery entry points; local unit tests that exercise recovery logic are isolated simulations only.

10. External dependencies and current read-only observations

10.1 Exact identity snapshots

VERIFIED_BY_EXECUTION, read-only public RPC: The reviewer queried chain ID, blocks, code, selected storage and view calls through a public BSC provider. The historical test block and a fresh sampled block were each pinned by number; a second provider later corroborated the beacon result at a newer pinned block. No account connection, signature or transaction was used.

Table 13. Pinned dependency snapshots

Snapshot	Block	Block hash	Mining owner	NFT beacon owner
Historical fork state, freshly reread	128,713,128	`0x760f050b7ec66ced2a2745918d7492650150f9d71d3159259412996ebe65081f`	zero	`0x68224f668083c29e9800be2a646d42d18cedf7e2`
New sampled state	130,348,784	`0xe82c9997ad7db34b24aed39e2ce784381207fd8f8967f2698683b9dcbbd04280`	zero	**zero**

A second public provider confirmed chain56 and the same hash at block130,348,784 but could not serve its old state (`missing trie node`). At its freshly selected block130,350,994, hash `0x565103b214800f27bd66c3ec13dce5d66c2cd0137c400b8838309ac35321791d`, it returned owner zero, raw owner slot zero and exactly the same beacon runtime. This corroborates the current beacon result across providers at different pinned blocks; it is not a second-provider verification of every dependency or of historical state. Evidence: `beacon-corroboration-result.json`, `beacon-secondary-result.json` and raw responses.

The following code identities matched at both snapshots and matched the retained identities. Exact addresses are supplied because immutable Pool bindings and external implementation semantics matter separately.

Table 14. Dependency addresses and runtime hashes

Component	Address	Bytes	Keccak-256 runtime
BEM	`0x5ce033B2bFCa3Af30b3e8C8457DeaF776A8b695a`	2,205	`0x982ba79538b51e54ee1382f57522098af0635b0f67f2574d393b29ed0958b344`
Mining proxy	`0x7E2E0DC66a3bD9103E69b766afA62d9f7b697b46`	130	`0xed6b710a769987f50494d713f278817992e3266e45898a990be878efc2fcbbce`
Mining implementation	`0xa3DBe873DA37cD4e4a13c7cEf23a7dB6Ca60f898`	24,574	`0x63b1124d3448fdeb28b163c52142e15a7a464d7ae7d817e51f6c242b62424479`
Mining linked library	`0xFbB661Fe281112C601954AbB0a3CDCC697a6E6bA`	10,136	`0x028ff888dc6b7505d507ffd33ef99d8541a23852d36a0463de235af6772e4737`
NFT proxy	`0xb1024b89886B9a34Aa4ff5F31C411D708b20a14C`	295	`0xd8c4b0216e0aadd615fbd134465b6af060a11769edc7c844d8f14d1b8a783992`
NFT beacon	`0xf8d6d8eb894d6971c8976ad8b4971cbefe028156`	619	`0x964014cd563c5bc62e1f8a3b2995969d6385f25da1e4efa353a01a62e0b75cca`

Component	Address	Bytes	Keccak-256 runtime
NFT implementation	0x8E1D125Def6d3826C278299273a0760D47626068	12,560	0x2ea433839e23b3773b7880d6f1807affb0f648ae98760b9a3b6394879665c345

BEM reports 8 decimals and `MAX_SUPPLY=2,100,000,000,000,000` raw units at both blocks. Supply is 39,430,105,671,950 raw at the historical block and 44,360,081,915,111 raw at the fresh sample. These are point-in-time view responses, not a formal proof that no token behavior can violate the Pool's assumptions. Mining implementation and NFT beacon/implementation bindings were read rather than inferred merely from code at the named addresses. Evidence: `protocol-snapshot.json`, saved requests/responses and proof source.

10.2 EX-01 - External immutability requires more than historical owner statements

Classification: Residual external dependency risk, with an informational correction to the historical authority assessment; not a confirmed current beacon-owner exploit. **Confidence:** High for observed bytes/storage and the small-beacon static conclusion; bounded for complete Mining/NFT/BEM semantics. **Actor/preconditions at risk:** A flaw or separately retained authority in a dependency, or future behavior outside the reviewed token/holder assumptions. The specifically hypothesized current nonzero beacon-owner upgrade route is **not supported at the fresh snapshot**.

Invariant: Pool NFT holder identity, Mining payee, external claim lifecycle and BEM conservation remain valid for permanent custody. **References:** Pool constructor `250-253,278-282`, holder checks `645-647,670-672`, `_readComposition:757-772`; external evidence above; `docs/MINING_LIFECYCLE.md:63-85,188-218`.

Unlike an arbitrary zero-returning owner getter, the exact observed 619-byte beacon was examined completely. Its only five selectors are ownership transfer, owner, renunciation, implementation and upgrade. Its only executable `SSTORE`s are at PCs `0x009d` (new owner), `0x012f` (renounce), and `0x01cc` (implementation). All are dominated by the guard at `0x020a` requiring `CALLER == uint160(SLOAD(0))`. Slot0 is zero at the fresh block; slot1 contains the observed implementation. No initializer, pending-owner path, arbitrary external/delegate call, creation or self-destruct route exists in the executable region. Bytes after `INVALID` at `0x235` are compiler metadata, not hidden functions.

A local opcode walk verified byte equality with the saved RPC result, length, selector/write locations and absence of those alternate instructions. The control-flow/owner-restoration conclusion is **SOURCE_CONFIRMED static bytecode reasoning**, not an executed mutation test. Under ordinary EVM account authentication and the observed code/storage, a nonzero caller cannot restore ownership or upgrade this beacon. A hypothetical RPC simulation allowing `from: zero` would not establish a signable real transaction; none was performed.

The state change is consistent with intervening renunciation, but its exact transaction/block/signer was not retrieved. The proof covers this beacon, not every function in the collection implementation or possible proxy-pointer modification. Code-hash equality establishes identity, not all semantic properties. Full NFT/Mining/BEM behavior and any other authority remain a professional release review gate. The full dependency tuple should be independently corroborated at the actual release block; this review corroborated the current beacon separately but did not duplicate every historical/code read across providers.

Impact if a remaining external assumption fails: NFT custody/payment destination or future admissions/harvest could fail without a production escape route. Already-held standard BEM can remain claimable independently if receipts and BEM still work. **Remediation/gate:** Retain a complete pinned dependency and authority attestation, obtain targeted independent bytecode/source review of remaining paths, and record explicit acceptance of permanent external dependency. Monitoring can detect divergence but cannot itself return an irrecoverably held miner. **Regression:** Isolated dependency fault models should show precisely which admissions/harvest/claims fail and which remain solvent. No live mutation is warranted.

10.3 EX-02 - Shared upstream cap changes the realized input vector

Severity: Medium conditional economic dependency; not a counterexample to the Pool's fixed-input theorem. **Confidence:** High for the conditional counterexample; real cap-boundary execution was not performed. **Evidence:** `VERIFIED_BY_EXECUTION` for independent stylized arithmetic; `AUTHOR_RECORDED` for the retained Mining cap/debt analysis; `SOURCE_CONFIRMED` for Pool claim ordering.

Actor/preconditions: A depositor chooses order near the remaining global mint cap, or unrelated permissionless claims consume the cap first. **Invariant/claim affected:** Same timestamp and nominal pending rewards alone imply admission-order equivalence. **References:** Pool `_admit:658-663`, `_collectCarried:712-725`; `docs/MINING_LIFECYCLE.md:127-131`; remediation report `181-197`.

The retained external analysis says Mining advances debt and mints `min(owed, remainingSupply)`, foregoing unpaid excess. At a fixed timestamp with remaining capacity105 BEM, suppose X(10:0) and Y(5:5) each are owed105 BEM. X-first realizes carries `(105,0)` and credits NAND100/LATCH0. Y-first realizes `(0,105)` and credits NAND50/LATCH50. A/B/C totals are unchanged because actual total is105. The actual per-miner vector differs. Old miners are also claimed before incoming miners, consuming shared capacity first if this branch is reached.

This scenario is not evidence that the current fresh supply is near exhaustion, and `pending()` is not assumed equal to actual claim. The independent Python cap fixture deliberately models the dependency; it does not execute deployed Mining.

Impact: Tail-emission category allocation and forgone upstream income can depend on ordering despite no time passage. Pool cannot distribute money it did not receive. **Remediation:** Preserve identical actual per-miner inputs and upstream state in every order-independence statement; independently validate cap exhaustion and debt consequences on an isolated simulation/fork. **Regression:** At fixed time, cap a local Mining model, record changed realized vectors, then separately assert Pool equality when the realized vectors are held fixed.

10.4 EX-03 - Known compiler exposure requires release-specific resolution

Severity: Low in the upstream advisory; unresolved release gate. **Confidence:** High for compiler/settings exposure, unknown for candidate-specific impact. **Evidence:** `VERIFIED_BY_EXECUTION` for fresh artifact metadata parsing; official advisory and `SOURCE_CONFIRMED` configuration analysis; `UNVERIFIED / NOT_EXECUTED` for candidate miscompilation and compiler differential exclusion. **Preconditions:** The affected optimizer receives the intermediate address/alias pattern and a runtime input realizes the alias. **Invariant:** Deployed bytecode faithfully implements the reviewed source semantics.

`foundry.toml:7-11` and fresh Pool, receipt and distributor metadata identify `0.8.24+commit.e11b9ed9`, Paris, `viaIR=true`, optimizer enabled with 200 runs, and no custom optimizer-step override. Version-specific compiler documentation confirms omitted step settings use defaults. Rebuilding

the same bytes with the same compiler authenticates the build; it does not establish compiler correctness.

The official SOL-2026-7 advisory, published 22 October 2026, describes stale address knowledge in Yul optimization. A resulting load or store can be wrong. Source inspection and passing tests cannot reliably exclude the intermediate-code/runtime conditions. Paris does not remove exposure. The 0.8.38 release announcement identifies the fix and the affected 0.8.19-0.8.37 range. This review found no concrete Groupmine instance of the faulty transformation.

Reproduction/evidence: Inspect the frozen configuration and the three fresh artifact `metadata.settings` records; compare against the published version/settings conditions. This reproduces exposure only. Neither an exact-version patched/unpatched compiler comparison nor a fixed-compiler behavioral/equivalence comparison was executed. Retrieving the complete tagged historical default-step source failed; the versioned documentation and advisory supply the default-setting evidence.

Remediation: Before release, obtain an exact-0.8.24 comparison with the narrowly backported fix, or prepare a separately frozen candidate using a selected fixed compiler starting with 0.8.38. The first option can establish non-impact if executable bytes agree under the advisory's comparison method; the second changes the reviewed build and requires regenerated identities, size/gas checks and behavioral review. Different bytes require investigation and do not themselves prove miscompilation; metadata and the Pool's embedded receipt runtime-hash constant must be accounted for. Changing exact pragmas or compiler settings in this target was outside the frozen-review instruction and was not done. **Regression:** Retain the compiler comparison, rerun accounting and authority tests, and reverify every generated/deployed artifact. A test-only comparison can detect differences but cannot prove equivalence for all inputs.

The current published known-bugs list was also checked. SOL-2026-5 is excluded by the via-IR pipeline. SOL-2026-4 and SOL-2026-2 require mutual internal recursion absent from the reviewed reachable production paths; external callbacks are separate EVM frames. No feasible SOL-2025-1 storage-slot-boundary layout was identified in the normal dynamic arrays. These are stated configuration/source exclusions, not differential proofs. Later-introduced SOL-2026-6, -3 and -1 do not include 0.8.24. Supporting evidence is `COMPILER_KNOWN_BUGS_REVIEW.md` and `compiler-known-bugs.json`.

10.5 Limits of retained Mining lifecycle analysis

The retained 133 checks and dispatcher/symbolic analysis are bounded evidence. They use 2-6 loop unrolls, syntactic memory/storage aliasing, unknown external returns, no reentrancy model, and manually/fork-classified write sites. Argument/error names are inferred; earlier implementation behavior is not covered. The report itself distinguishes Pool-side revocation-payout booking reasoning from a real Pool-held revocation execution. These limits remain even though the sampled current code hashes match.

The Pool's strict skip requires REVOKED to be terminal and unpaid future claimability absent. The retained lifecycle analysis/fork observations support that claim for the named implementation; they do not prove every possible dependency behavior. Likewise the actual per-miner balance delta assumes one `Mining.claim(key)` pays the corresponding key's reward rather than mixing unrelated payouts. This review's fresh protocol-fork replay was blocked before process creation. The separately completed public read-only snapshots establish identities and named state values, not those lifecycle semantics. No public-chain mutation was attempted to obtain stronger evidence.

11. Intended policies and remaining operational risks

These are specified behavior or setup boundaries, not silently relabeled code exploits. Each needs explicit acceptance because owner-zero permanent custody removes later correction paths. Confidence is high for the source-defined policies and the off-chain verification boundary; the truth and completeness of the actual launch allocation remain UNVERIFIED. Their materiality is operational/economic acceptance, not an additional unexpected production vulnerability severity.

Table 15. Intended policies

Policy ID	Rule, actor/precondition and impact	Evidence / references	Required treatment and regression
POL-01	REVOKED miners retain historical burned-material weight. With aggregate1:9, revoking the LATCH-only miner leaves future105 BEM from surviving NAND production allocated10/90 to participants. Registry work remains too.	SOURCE_CONFIRMED: Pool `692-693,780-788`; `ARCHITECTURE.md:179-186`. This is the specified historical-material model, not active-production weighting.	Accept the economic model explicitly; preserve mixed-category revocation and later ordinary-income tests. Do not remove weight as a casual availability fix.
POL-02	The 1-16 depositor whitelist never rotates. Loss/inaccessibility of every authorized account stops future additions while existing claims/harvest/releases can continue.	SOURCE_CONFIRMED: Pool `267-276,627-631`.	Select durable authorized operational wallets before deployment; test nonlisted rejection and post-renunciation listed access. No owner backdoor should be inferred.
POL-03	Plain NFT transfer/delayed registration is unsupported and recovery is not promised. Revocation before first composition can lock new rewards; with a composition, unregistered payouts become ordinary income at recognition.	SOURCE_CONFIRMED: Pool `345-355,511-516,655-694`; SALUS #1.	Use atomic deposit only; keep explicit unsupported-path documentation and test unsolicited assets do not populate the registry or rewrite past claims.
POL-04	Premature renunciation, incorrect immutable bindings, final beneficiary mistakes and incapable recipient contracts can be irreversible. A/B role holders can migrate their own unpaid role, but receipt beneficiaries have no administrator redirect.	SOURCE_CONFIRMED; fresh premature-renounce test; SDK receiver checks `pool.ts:268-301`; CLI renounce gates `463-478`.	Finalize/review before renouncing; test actual wallet receipt acceptance, outbound transfer and Pool claim, including guards/delegation. Generic ERC-165 success is insufficient.
POL-05	Contributions were received through an EOA; the operator supplies the final reviewed JSON. Finality proves the committed on-chain state, not the truth of contribution accounting or participant consent.	Business brief; SOURCE_CONFIRMED allocation/verification boundary, ST-02.	Independently reconcile the final JSON, contributor approvals and full receipt coverage; preserve its hash/provenance and rehearse actual allocation size before irreversible finalization.

For POL-01/02/03, the risk is material operational/economic acceptance rather than an unexpected violation of the specified model. POL-04/05 are material setup-control requirements. The proper response is concrete release evidence and informed acceptance; introducing a production sweep/withdrawal or owner override would change the agreed custody model and require a new review.

12. Fresh execution ledger and evidence limits

12.1 Reproducible environment and isolation

The execution copy came from `git archive` of the exact detached target. Node 24.22.1, npm 11.9.0, Foundry 1.4.3 (`fa9f934bdac4bcf57e694e852a61997dda90668a`) and native solc `0.8.24+commit.e11b9ed9` were used. The Foundry release archive SHA-256 was `325ba04dc5cb41c110723b00ac291f8269f8cd785028299aad8252ef980961a7`; the compiler binary SHA-256 was `fb03a29a517452b9f12bcf459ef37d0a543765bb3bbc911e70a87d6a37c30d5f`. Dependencies came from the frozen lockfile, including OpenZeppelin 5.4.0, viem 2.57.2, zod 3.24.2, tsx 4.19.3 and TypeScript 5.7.3.

`npm ci --ignore-scripts --no-audit --no-fund` installed 23 packages. Distinct empty temporary npm user/global configuration files avoided loading user configuration. Root lifecycle hooks were absent; dependency install hooks remained disabled. The offline Forge campaigns received an explicit minimal environment and local compiler path. No generic repository audit/deployment wrapper was assumed safe merely because its name suggested testing.

Before enabling FFI for the two differential groups, the exact Python programs and invocation sites were inspected. They parse generated numeric traces, calculate in memory, and print results; imports are `sys` and `fractions.Fraction`, with no network, filesystem reads or subprocess execution. FFI used `python3 -I`. The inspected paths and hashes are in `execution-safety.json`.

The executed local settings were Paris, optimizer 200, via-IR. The prepared fork command would have used the repository's Cancun harness profile and loaded the separately built Paris production artifacts from `out`. It was blocked before process creation, so no fresh fork compilation or behavioral test is claimed. The proposed command used a public endpoint and in-process simulations, with no transaction submission. Commands and explicit selected test files are retained in the result JSON, approval-block record and runner scripts.

12.2 Fresh check outcomes

These outcomes are this review's execution, not copied PASS labels from the author. Existing tests remain author-written tests even when independently rerun. Passing a regression that asserts a weakness confirms that weakness; it is not evidence that the weakness is fixed.

Table 16. Fresh execution outcomes

Check	Fresh result	Scope and evidence
Source-only Solidity build, sizes and build metadata	PASS	Exact compiler/settings; production and generated canary source compiled. `build-production-result.json`, `build-production.log`.
Main original Solidity campaign	**226/226 PASS; 0 failed, 0 skipped**	117 unit tests (including 12 canary-recovery cases), 45 SALUS examples/fuzz tests, 15 remediation tests, 30 fee-audit tests and 19 invariant properties. `tests-primary-*-result.json`, `forge-fresh-summary.json`.
Prior-audit Solidity group	**10/10 PASS**	Includes the 400-run differential FFI property. `tests-prior-audit-result.json`.

Check	Fresh result	Scope and evidence
SALUS differential FFI group	**2/2 PASS**	300-run differential property and acceptance example; `tests-salus-ffi-result.json`; separate from the primary 226.
New independent Solidity regressions	**11/11 PASS**	Five accounting/API and six authority/finality tests; described below. `tests-independent-result.json`.
Narrow invariant coverage confirmation	**1 repeated property PASS**	A repeated property for handler nonvacuity, counted separately rather than added to the primary 226. `tests-coverage-confirmation-result.json`.
Original protocol-fork campaign	**NOT_EXECUTED: 0/20 started**	Automatic approval review blocked the prepared command before process creation; no fresh fork observations.
Original SDK TypeScript tests	**78/78 PASS; 0 failed, 0 skipped**	Ten files, run separately. `sdk-results.json`, `sdk-*.test.log`.
Prior-audit and fee-consumer TypeScript tests	**13/13 PASS**	Revert matcher 9, supply-gap verification 1, fee consumers 3. `audit-typescript-loopback.log` and result.
TypeScript typecheck	PASS	`tsc --noEmit`; `typecheck-result.json`.
Independent arithmetic oracles	PASS	60,480 small-state cases, 2,000 fixed-input permutation/partition comparisons and 18,000 random-trace states. Python arithmetic, not Solidity execution. Seed 6345; `independent-arithmetic.json`.
Per-receipt threshold propagation	PASS, counterexample confirmed	Independent Python check of the S_N=20 example in PC-01; `receipt-threshold.json`.
SDK deployment-identity fixture	PASS, checker gaps confirmed	Actual frozen SDK with an in-memory read client; omitted distributor and inconsistent masked slots accepted as described in TL-01/02. `identity-review.json`.
ABI and bytecode export reproduction	PASS	All six entries and both complete generated files reproduce from fresh artifacts. `abi-build-review.json`, `bytecode-export-review.json`.
Provenance / preserved-history comparisons	PASS	Target, source fingerprints, SALUS appendix identities and protected historical files independently compared; `provenance-hashes.json`.
Public dependency and beacon observations	PASS within the stated read scope	Exact block/state/code results and second-provider qualifications in section 10; `protocol-snapshot.json`, `beacon-*-result.json`.
Beacon opcode structure and compiler metadata	PASS as structural checks	Bytecode/source reasoning remains explicitly separate from local mutation tests or compiler-correctness proof; `beacon-structural-check.json`, `compiler-known-bugs.json`.

The SDK breakdown is allocation 6, canary metadata 7, canary revert 6, canary 4, chain/binding CLI 4, gas 11, math 9, plans 10, read layout 8 and verification 13. The canary tests exercised local logic, source generation in check mode and simulated revert handling. They did not invoke a live canary workflow.

The 11 new Solidity tests independently checked the four admission-order/partition forms of the acceptance example and later ordinary income; same-ratio checkpoint thresholds;

carried-versus-ordinary threshold differences; a rational shortfall exceeding one raw unit; duplicate-preview versus actual-payment behavior; a lying owner-zero distributor retaining delivery power; genuine empty finalization opening custody; ownership rotation separating retraction authority from inventory; unsolicited genuine receipt inventory blocking finalization and activation; premature receipt renunciation permanently blocking activation; and renunciation clearing pending ownership without a restoration route. Exact test names, outcomes and gas observations are in the retained result JSON. Some tests deliberately use mock dependencies to isolate a production boundary and do not establish real-protocol feasibility of every numeric input.

12.3 Fuzzing, invariants and positive-path coverage

The original primary campaign contains three fuzz properties at 512 runs each: accounting math, cumulative fee releases, and carried-input order/partition equality. The main invariant configuration is 256 runs, depth 64, `fail_on_revert=true`; the pinned campaign seed is `0x6345de8`. Each of the 17 main/coverage invariant properties reported 16,384 handler calls and zero uncaught reverts. The two finality properties reported the same run/depth totals. Eight of the 17 main/coverage properties are inherited repeats. Identical seed/handler count profiles are not claimed to be independent novel traces, and iterations/calls are not counted as additional test cases.

An invariant handler can swallow expected failures or do no work even when its test passes. Therefore zero uncaught reverts alone was not accepted as positive-path coverage. The fresh finality CSV contains 504 observed rows, 258 with successful admissions and 352 with refused admissions; the respective recorded totals are 1,740 and 1,839. These are repeated campaign rows, not 504 distinct test cases or a guarantee of every lifecycle state. The separate rerun of `PoolInvariantCoverageTest.invariantRecognizedTotalMatchesModel` produced 214 recorded rows: 211 included successful admissions and 81 included rolled-back admissions, totaling 498 successful and 166 rolled-back admissions across those rows. Both paths were therefore nonvacuously exercised. The observed row counts were retained instead of assuming every configured run necessarily emitted a row.

Protocol-fork scope: the prepared command selected the original 20 tests at the retained principal block 128,713,128, with tests that also select historical block 119,597,280. The public endpoint was `https://bsc-mainnet.public.blastapi.io`; no API token was used. Automatic approval review rejected the escalation because the explicit tool PATH referenced restricted runtime directories under `/opt/codex`, and instructed against a workaround. The review stopped that check. Zero fork tests executed, zero fresh fork observation files were produced, and the retained behavioral fork reports remain AUTHOR_RECORDED. This is a permission limitation rather than a failed contract assertion. The requested command and rejection are preserved in `tests-protocol-fork-approval-block.json`; running it requires approval of the blocked runtime access.

12.4 Failed attempts and how they affect confidence

Table 17. Failed attempts and limitations

Attempt / limitation	Actual outcome	Treatment
Initial repository acquisition in the restricted environment	Public clone could not reach the configured proxy; a web target fetch was also unavailable.	Authorized read-only network retry acquired the exact public repository. No alternate target was substituted.

Attempt / limitation	Actual outcome	Treatment
Initial npm configuration isolation	Using `/dev/null` as both user and global config caused npm's duplicate-config error.	Retried with two distinct empty temporary files; locked installation completed with scripts disabled. The failed attempt was setup work, not a failing application test.
Monolithic `forge build --sizes`	solc was terminated by SIGKILL after about 724.47 seconds while compiling 104 files.	Cgroup diagnostics show an 8 GiB limit, an approximately 8 GiB peak and an OOM kill; this supports memory pressure, without claiming per-PID kernel attribution. Source-only build and selected test groups were then compiled serially with unchanged sources/settings. `build-result.json`, `cgroup-memory-diagnostic.json`.
Initial TypeScript audit tests in the restricted sandbox	Loopback HTTP/Anvil listeners failed with `EPERM`; process-mode reporting showed 2/3 file tests, and the diagnostic in-process run showed 5/13 tests.	Retained both failed logs. An authorized retry permitting local listeners passed all 13. These failures were not silently replaced or counted as application defects.
Initial Node public-RPC query	Fetch failed with `EAI_AGAIN`.	Explicit use of the configured environment proxy completed the read-only snapshot. Both logs are retained.
Preparatory manually encoded RPC block probe	A hexadecimal block value selected 128,733,096 rather than 128,713,128.	That probe is retained but excluded from exact-pin identity claims. The request was regenerated from the correct integer and the correct block/hash recorded.
Second public provider at block 130,348,784	Chain and block-hash queries succeeded, but historical state/code queries returned `missing trie node`.	No historical state corroboration was claimed from those errors. A separately pinned later block 130,350,994 corroborated beacon owner zero, raw owner slot zero and identical runtime.
Parallel operational immutable-checker proof	An automatic safety filter interrupted the delegated work before a complete patched-runtime EVM demonstration.	The operational demonstration remains NOT_EXECUTED. Pure local SDK comparison/schema probes and static compiled-occurrence analysis were completed instead; TL-02 states that boundary.
Exact historical compiler optimizer source retrieval	Tagged source retrieval failed; a direct raw request was denied by the environment.	Official versioned documentation, current advisory and parsed metadata establish settings exposure; no complete historical step string was claimed.
Protocol-fork attempt	Automatic approval review rejected restricted `/opt/codex` runtime access before process creation.	No workaround or retry was attempted. All 20 intended tests remain NOT_EXECUTED, and no fresh behavioral fork result is claimed.

The small number of environment/retrieval corrections above are not new production findings. They are disclosed because they change what was actually verified. The complete monolithic build was not rerun after the narrower successful compilation strategy; that is a resource-bound execution choice, not a modification of the frozen implementation.

12.5 Author evidence retained without promotion to fresh proof

The author records 226 primary Solidity tests, 10 prior-audit Solidity tests, 2 SALUS FFI tests, 78 SDK tests, 13 audit TypeScript tests, 20 fork tests, 59 local end-to-end assertions, 25 fork end-to-end assertions, 266 Safe workflow checks and 46 local canary assertions. Fresh reruns of a corresponding suite are identified above; they do not independently rerun the other workflows. The 11 GasModel cases already belong to the primary count. The 63 rows in `assertions.json` are requirement/report rows, not 63 extra executable tests.

The retained source fingerprint matches the frozen source even though the recorded author run began with 58 dirty paths against its then-baseline. This resolves a provenance question; it does not authenticate every recorded execution claim. The author manifest's overall `INCOMPLETE` result and omitted Slither gate remain relevant. Historical mainnet canary transactions with controlled assets do not prove this candidate's accounting or setup safety.

12.6 Checks not executed or not established

- **Slither:** unavailable in the environment and not run. A successful compiler/test run is not a substitute for that declared release gate.
- **The 20 protocol-fork tests:** prepared and requested, but blocked by automatic approval review before execution as described above. Separate read-only RPC identity/state observations completed successfully.
- **Full local/Safe/fork end-to-end workflow:** the retained 59/25/266-assertion workflows and 46-assertion canary workflow were not independently rerun. Their reports remain AUTHOR_RECORDED.
- **Live canary, deployment, approval, preparation, recovery or public signing/broadcast:** deliberately not executed under the review boundaries. Isolated unit recovery tests do not contradict this restriction.
- **Actual launch allocation and wallets:** no final contribution JSON, final beneficiary set/guard configuration or largest actual per-wallet allocation was supplied. Full-round business correctness and final recipient usability therefore remain unverified. A recorded 1,500-receipt sample is not proof for an unknown final allocation.
- **Real-protocol admission at hundreds of registered miners:** not established by the 24-miner retained sample or the fresh 400-miner mock benchmark. A numerical extrapolation is not an executed capacity test.
- **Real cap-exhaustion debt behavior and extreme rounding inputs:** the cap counterexample is an independent stylized model; the uint32 burn counterexample is a contract-domain proof. Neither establishes attainability of those cases in the deployed Mining protocol.
- **Complete Mining/NFT/BEM verification:** no comprehensive formal proof or complete professional external-dependency audit was performed. The beacon proof is deliberately limited to its observed runtime/state; the precise intervening renunciation transaction was not retrieved.
- **Operational inconsistent-immutable demonstration:** the complete patched-runtime EVM fixture was not executed. Acceptance by the actual comparison function and the real artifact's distinct getter/operational occurrences are established separately.
- **Compiler-bug exclusion:** no exact-version backported-fix build, fixed-compiler candidate build, whole-program equivalence proof or all-input differential analysis was executed. SOL-2026-7 remains a release gate.

13. Minimal correction priorities and release gates

13.1 Corrections to prepare in a new candidate

The frozen implementation remains unchanged. The following are specific recommended changes or release conditions; they are not claims that fixes were applied.

Table 18. Prioritized corrections

Priority	Required action	Evidence needed for closure
1 - deployment identity	Make the bound distributor part of every complete deployment attestation. Read `pool.distributor()` and require agreement with the manifest; authenticate its genuine runtime, receipt/pool links and irreversible state. Preserve immutable-reference groups and verify every occurrence against its expected constructor value.	Negative tests for missing/mismatched distributor, lying zero-owner views, unreviewed code, repeated-immutable disagreement and upgradeable look-alikes; positive exact-runtime deployment test. A partial result must not be called unqualified `MATCHES`.
1 - compiler	Resolve EX-03 through the exact-version fix comparison or a separately frozen fixed-compiler candidate.	Retained compiler provenance/comparison, analyzed differences, regenerated receipts/Pool/distributor artifacts, size/gas results and rerun functional/authority tests. Do not silently substitute new bytecode under this review's SHA.
2 - accounting specification	Replace absolute rational/fixed-timestamp/checkpoint-invariance wording with the exact fixed-realized-input theorem and finite-precision bounds. Explicitly accept the small category threshold and per-receipt residue effects, or implement a separately specified stronger rounding design.	Independent rational and implementation-precision regressions including PC-01's two counterexamples and receipt threshold propagation; documentation and SDK agree on raw units, rounding and period boundaries.
2 - setup finality / full round	Decide whether full-round funding/allocation is an on-chain invariant or a strictly enforced deployment gate. Empty/partial finality must not be presented as proof of full participation.	Final reviewed JSON, contributor reconciliation, full coverage and recipient transfer/claim rehearsal, followed by pinned on-chain verification before any irreversible renunciation/custody. Test premature renunciation and all reachable setup orders.
2 - admission availability	Choose and document an operational registry limit with measured headroom; independently characterize persistent old-miner failure and REVOKED behavior.	Real-protocol measurements at the intended maximum and batch shape, under the current transaction gas limit; bounded failure recovery/acceptance plan. Paginating settlement across transactions requires a new accounting specification and review.
3 - raw preview	Deduplicate IDs or explicitly specify multiplicity in `claimableBatch`, with consistent SDK behavior.	Duplicate/mixed-duplicate tests show preview semantics agree with the documented executable claim.

Priority	Required action	Evidence needed for closure
3 - SALUS #4-#6	Fix or explicitly accept bounded revocation capacity, ownership/inventory handover and unsolicited receipt-funding liveness.	Largest actual wallet revoke test; handover with top IDs; outside unpaid receipt funding before finalize/activate. A strict inventory plan should eliminate the finalization-grief precondition before launch.
3 - dependency attestation	Make `protocol-check` chain-aware and complete for the intended dependency tuple; pin all reads to one block and qualify scope.	Current Mining proxy/implementation/library, BEM, NFT proxy/beacon/implementation identities and relevant owners/authorities, independently corroborated for release. Document any still-unproved semantic assumptions.
Separate canary work	Address missing binding event and duplicated ID constants in the canary family if that family remains in use.	Regenerate and retest canary code. Preserve the explicit recovery controller and controlled-asset scope; do not transfer canary assurance to production.

13.2 Concrete pre-custody evidence required

Before the first production miner enters permanent custody, a release package should contain the corrected/final candidate SHA, reproducible compiler/dependency identities, full generated-artifact/deployed-bytecode comparison, all constructor/binding addresses, receipt supplies and owner/pending-owner state, distributor funding/finality/allocation state, the final allocation JSON and its review provenance, actual beneficiary receiver/transfer/claim results, immutable whitelist and role-recipient approval, and realistic admission/allocation gas results. These should refer to one candidate and named chain/block, not a mixture of historical runs.

Run the missing declared static-analysis gate or explicitly replace it with a documented reviewed alternative. Complete the intended end-to-end setup and role/recipient workflows on an isolated environment using the actual launch configuration. Obtain an independent professional reassessment of the production changes, compiler resolution, remaining SALUS findings and material external dependencies before accepting permanent custody. This review is substantive internal evidence, but it does not change the SALUS PDF's Pending statuses or grant third-party audit sign-off.

Release should also explicitly accept the intended policies: revoked miners retain historical burn weight; the depositor whitelist never rotates; direct transfers and pre-composition anomalies have no promised rescue; genuine setup renunciation is irreversible; and finalized distribution reflects reviewed off-chain input rather than proving its truth. Monitoring of dependencies, gas and income can expose problems, but it cannot add a production withdrawal path after deployment.

13.3 Decision

The remediation materially improves carried-income attribution and setup gating, and extensive fresh bounded execution supports those changes. It does not justify an unconditional PASS for the frozen release. The unresolved deployment-verifier guarantees, current compiler exposure, remaining setup findings and candidate-specific deployment/dependency evidence are concrete reasons to hold production custody while preparing a new, reviewable release package.

Appendix A. Evidence bundle and reproduction map

The accompanying `GROUPMINE_6345de8_REVIEW_EVIDENCE.zip` contains the review's fresh logs, result JSON, independent proof sources, selected analysis notes, exact command runners, source-preservation evidence and a SHA-256 manifest. It omits installed dependencies, compiler binaries, repository duplication and stale preliminary review notes. The original frozen public repository remains the source for existing tests and implementation files. `README.md` inside the bundle explains paths and replay boundaries.

Table 19. Evidence reproduction map

Evidence question	Principal files in the bundle
Exact target / author-record linkage / source unchanged	`logs/target-identity.txt`, `analysis/provenance-hashes.json`, `logs/source-preservation-final.json`
New accounting or authority reproductions	`proofs/InternalAccountingReview.t.sol`, `proofs/InternalAuthorityReview.t.sol`, `logs/tests-independent-result.json`
Independent rational versus finite-precision arithmetic	`proofs/independent_accounting.py`, `proofs/receipt-threshold.py`, `logs/independent-arithmetic.json`, `logs/receipt-threshold.json`
Complete fresh suite outcomes, seeds and commands	`logs/forge-fresh-summary.json`, `logs/tests-*-result.json`, `logs/sdk-results.json`, `logs/audit-typescript-*.log`, `runners/`
Deployment checker deficiencies	`proofs/identity-review.ts`, `logs/identity-review.json`, `analysis/IMMUTABLE_IDENTITY_REVIEW.md`, `analysis/immutable-build-review.json`
Fresh dependency state and narrow beacon proof	`proofs/protocol-snapshot.ts`, `logs/protocol-snapshot.json`, `logs/beacon-*`, `analysis/BEACON_REVIEW.md`, `analysis/beacon-structural-check.json`
Compiler exposure and fresh generated artifacts	`analysis/COMPILER_KNOWN_BUGS_REVIEW.md`, `analysis/compiler-known-bugs.json`, `analysis/abi-build-review.json`, `analysis/bytecode-export-review.json`
Environment failures and restricted/unexecuted boundaries	`logs/build-result.json`, `logs/cgroup-memory-diagnostic.json`, failed TypeScript/RPC logs, `logs/execution-safety.json`, this report section 12

Reproduction must use an isolated copy at the exact SHA with the same dependency lock and compiler settings. The runner files contain absolute paths from this session; adapt only those workspace/tool paths when replaying. Do not treat a replay instruction as permission to access credentials, execute a deployment wrapper or broadcast transactions.